

五、机器人仿真实验指导

1. 实验描述

在测试和开发的过程中，往往需要频繁对设计的内容进行修改，而且为了验证某个具体的功能，还需要保持其他条件不变。在真实的环境中进行测试时，尽管效果更为真实，但是也带来了效率低下，操作繁琐等问题。

为了实现高效快捷的开发和测试，就需要使用仿真的方法，通过在仿真软件中运行指定的功能，可以快速方便稳定的测试功能执行的效果，如果仿真无误则可以在真实环境中再进行测试。先仿真再实测也是开发过程中经常使用的方法。

本次课我们要学习和操作的便是在仿真环境下进行建图和自主导航的功能，通过仿真运行，可以检测功能执行的理论效果。

2. 实验目标

- (1) 认识 ROS 中的二维仿真平台 STDR;
- (2) 理解机器人仿真的相关配置文件及运行逻辑;
- (3) 掌握仿真环境下，执行定位、建图、导航的相关操作。

3. 实验步骤

步骤 1：仿真器介绍、配置及实操

(1) 配置逻辑

智能机器人的仿真实验是基于机器人内部的软件平台，仿真平台名称为 STDR，全称为 Simple Two Dimentional Robot Simulator，即简易二维机器人模拟器（也称为仿真器），它可以非常容易的对在二维平面移动的机器人进行仿真，STDR 的设计目的目的是为了尽可能的简单的去模拟单个移动机器人的运动或者多个机器人的协同仿真。而且 STDR 不仅可以运行在一个有图形界面的环境中，还可以运行在一个无桌面环境的系统中，这样我们就可以在终端中使用 ssh 连接后就可以进行仿真任务了。

STDR 可以完美的与 ROS 兼容配套，机器人上的传感器数据都是通过 ros 话题的形式发布。跟 ROS 运行一样，STDR 的运行也可以将图形用户接口和服务器分开在不同的电脑上

运行，而且 STDR 也可以和 RViz 一起工作，在 RViz 中将 STDR 中的机器人数据可视化显示。

智能机器人的全部仿真程序都会以 STDR 仿真器作为基础，在工程文件中，STDR 仿真器的配置文件夹路径为 `src\sebot_simulator\launch`，具体文件如下。

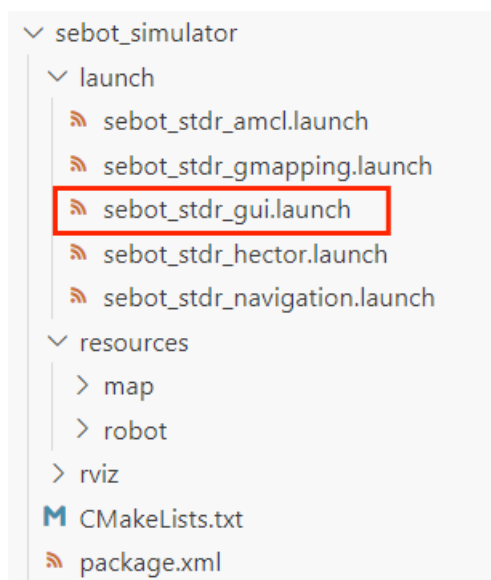


图 5-1 STDR 仿真器配置文件

其内容如下所示，文件中的配置逻辑为

- ① 启动 STDR 仿真器，此仿真器是已经安装完成的，可直接进行调用，运行时并无图形界面，需要另外设置；
- ② 加载需要使用到的地图文件，此地图将会影响到后续的仿真；
- ③ 设置地图的静态坐标系；
- ④ 启动 STDR 图形化界面，此步骤用于显示 STDR 仿真器的图形界面，便于观察效果；
- ⑤ 加载机器人物理模型，此步骤是在 STDR 仿真器中显示对应的机器人模型并加以控制。

```
<launch>

  <!-- 启动 STDR 模拟器 -->

  <include file="$(find stdr_robot)/launch/robot_manager.launch" />

  <!-- 加载 STDR 地图 -->

  <node type="stdr_server_node" pkg="stdr_server" name="stdr_server"
output="screen" args="$(find
sebot_simulator)/resources/map/sebot_map.yaml"/>
```

```

<!-- 发布地图静态坐标系 -->

<node pkg="tf" type="static_transform_publisher" name="world2map"
args="0 0 0 0 0 0 world map 30" />

<!-- 启动 STDR 图形化界面 -->

<include file="$(find stdr_gui)/launch/stdr_gui.launch"/>

<!-- 加载智能机器人物理模型 (x y th) 机器人起始位置-->

<node pkg="stdr_robot" type="robot_handler" name="$(anon
robot_spawn)" args="add $(find
sebot_simulator)/resources/robot/sebot_robot.yaml 0.5 0.5 1.57" />

</launch>

```

(2) 实操说明

配置文件中可以进行修改的部分为“加载 STDR 地图”和“加载智能机器人物理模型”。

地图文件夹为 `sebot_simulator/resources/map`，修改地图文件名时，直接在配置文件中修改地图配置文件的名称即可，注意地图配置文件和地图图片文件一致，可使用实际建好的地图，也可自行绘制地图并编写配置参数，最终将地图的图片文件和配置文件放置于 `sebot_simulator/resources/map` 文件夹下。



图 5-2 地图文件所在路径

```

src > sebot_simulator > launch > sebot_std_gui.launch
1 <launch>
2
3 <!-- 启动STDR模拟器 -->
4 <include file="$(find stdr_robot)/launch/robot_manager.launch" />
5
6 <!-- 加载STDR地图 -->
7 <node type="stdr_server_node" pkg="stdr_server" name="stdr_server" output="screen" args="$(find sebot_simulator)/resources/map/sebot_map.yaml"/>
8
9 <!-- 发布地图静态坐标系 -->
10 <node pkg="tf" type="static_transform_publisher" name="world2map" args="0 0 0 0 0 world map 30" />
11
12 <!-- 启动STDR图形化界面 -->
13 <include file="$(find stdr_gui)/launch/stdr_gui.launch"/>
14
15 <!-- 加载商用机器人物理模型 (x y th) 机器人起始位置 -->
16 <node pkg="stdr_robot" type="robot_handler" name="$(anon robot_spawn)" args="add $(find sebot_simulator)/resources/robot/sebot_robot.yaml 0.5 0.5 1.57" />
17
18 </launch>

```

图 5-3 地图文件配置

需要注意此步骤配置的地图会影响到后续定位、建图及导航功能中加载的地图，后续调用此文件时要注意配置的地图文件。

设置机器人模型时，需要分别添加机器人模型的配置文件以及机器人在 STDR 仿真器中的初始位姿。

机器人的模型配置文件路径为 `sebot_simulator/resources/robot`，配置文件名为 `sebot_robot.yaml`，此配置文件基于真实的智能机器人模型，设置了机器人的轮廓、激光雷达以及超声波传感器的相关数据。此配置文件可结合自身需求进行修改，如果没有特殊需求，建议不做修改。

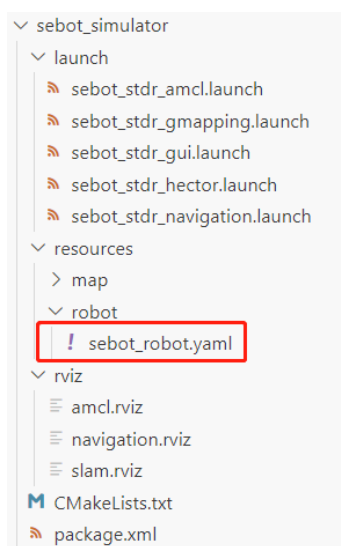


图 5-4 机器人模型配置文件

仿真器配置文件中模型配置文件后面表示机器人的初始位姿参数，分别为 x 坐标、 y 坐标、朝向，坐标以米为单位，左下角为坐标原点，向右为 x 轴正方向，向上为 y 轴正方向，朝向单位为弧度，向右为 0 弧度，逆时针为正向旋转方向。根据此标准可以调整机器人的初始位姿状态。详细效果可参考后续配图。

```

src> sebot_simulator> launch> sebot_std_gui.launch
1 <launch>
2
3 <!-- 启动STDR模拟器 -->
4 <include file="$(find stdr_robot)/launch/robot_manager.launch" />
5
6 <!-- 加载STDR地图 -->
7 <node type="stdr_server_node" pkg="stdr_server" name="stdr_server" output="screen" args="$(find sebot_simulator)/resources/map/sebot_map.yaml"/>
8
9 <!-- 发布地图静态坐标系 -->
10 <node pkg="tf" type="static_transform_publisher" name="world2map" args="0 0 0 0 0 world map 30" />
11
12 <!-- 启动STDR图形化界面 -->
13 <include file="$(find stdr_gui)/launch/stdr_gui.launch"/>
14
15 <!-- 加载服务器机器人物理模型 (x y th)机器人起始位置-->
16 <node pkg="stdr_robot" type="robot_handler" name="$(anon robot_spawn)" args="add $(find sebot_simulator)/resources/robot/sebot_robot.yaml 0.5 0.5 1.57"/>
17
18 </launch>

```

图 5-5 机器人模型配置及位姿设置

有关仿真的实操实验均基于智能机器人进行。启动机器人，在终端中输入以下命令启动 STDR 仿真器，然后即可查看对应的地图信息和机器人模型信息。

```
roslaunch sebot_simulator sebot_std_gui.launch
```

STDR 仿真器中显示的地图与加载的地图完全一致，左侧为信息显示：第一部分是二维地图的实际宽和高尺寸以及每个像素格表示的实际距离；第二部分是机器人的基坐标系。右侧为仿真界面，蓝色圆圈表示的是机器人的轮廓，红色线条表示激光雷达的点云数据，灰色三角表示未检测到障碍物信息的超声波传感器，绿色三角表示检测到障碍物信息的超声波传感器。机器人的初始位置决定于配置文件中设置的位姿信息。

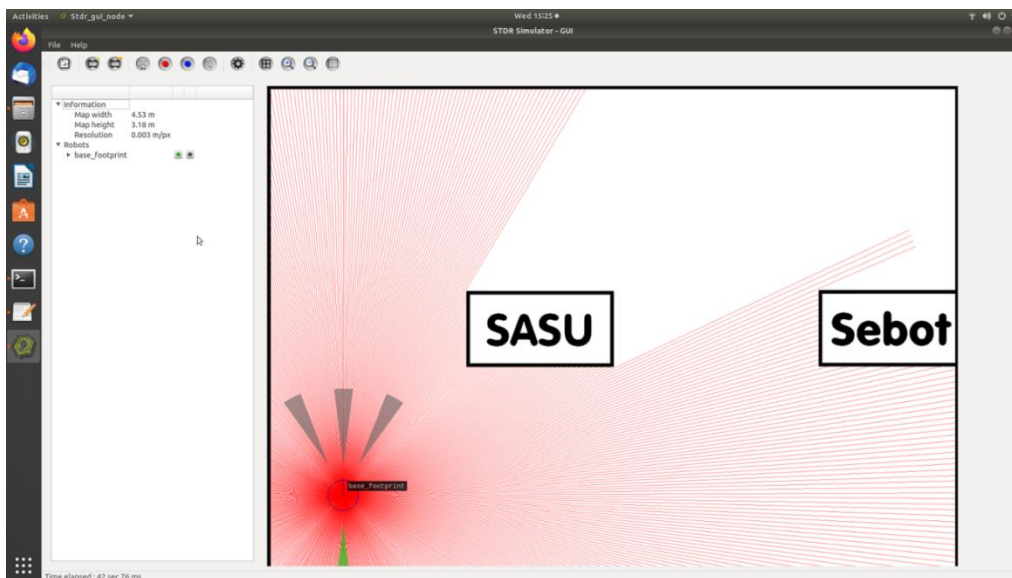


图 5-6 位姿参数为 0.5 0.5 1.57 的启动界面

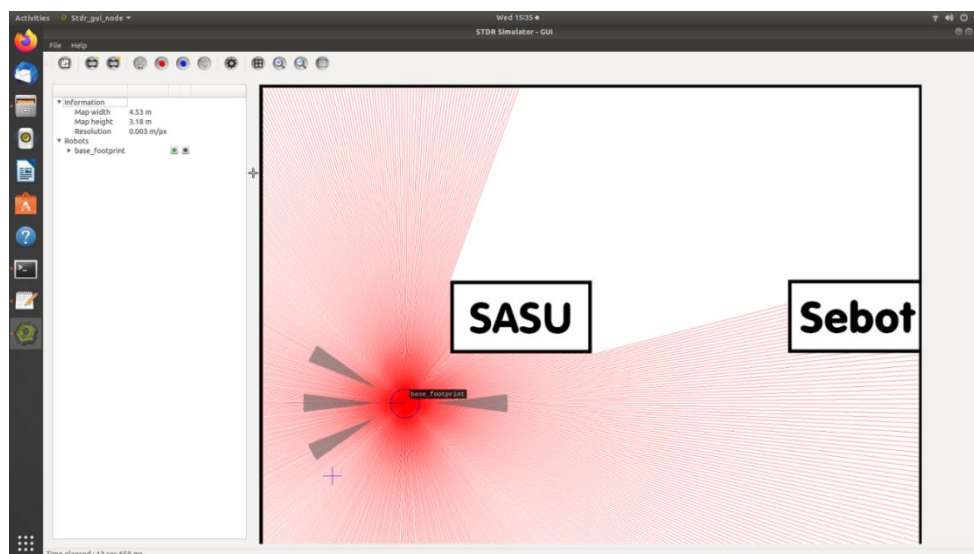


图 5-7 位姿参数为 1 1 3.14 的启动界面

加载不同地图时可根据需要调整仿真机器人的初始位姿。更换完新地图后，需要进一步确认机器人在地图中的初始位置，若该地图不存在当前位置坐标，程序未能启动并报错；若仿真机器人初始位置在封闭的地图外，无法遥控其进入地图。

(3) 自定义地图

如果想要自行绘制地图并编写相关的配置文件，可参考以下说明。

① 自定义地图参数：

例如，实际要搭建的地图尺寸及形状如下：

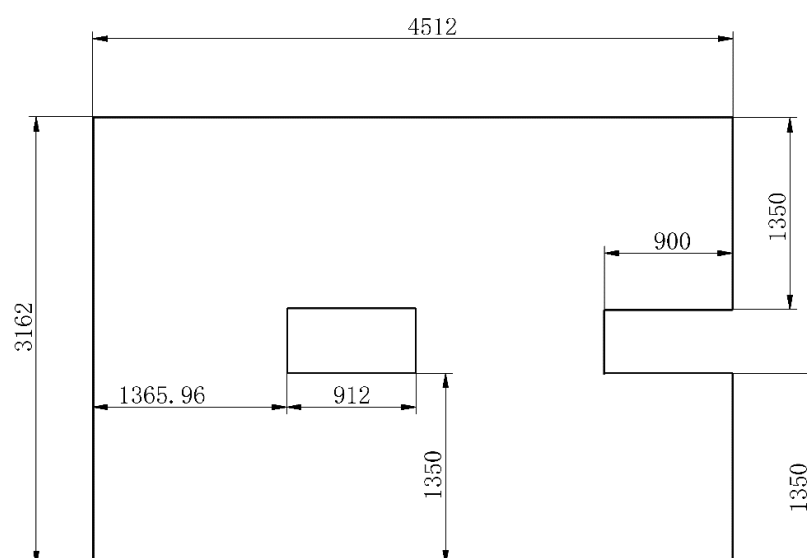


图 5-8 示例地图图纸

实际地图由长 45cm，高 35cm 的挡片拼接构成，拼接成的总体轮廓长 4.512m（ $10 \times 45 + \text{卡扣}$ ），宽 3.162cm（ $7 \times 45 + \text{卡扣}$ ）。

“yaml”文件中按分辨率 $\text{resolution}=0.003$ （米/像素），绘制实际地图的仿真地图，分辨率选择越高，仿真地图更清晰，若地图面积较大，推荐减小分辨率。

首先计算实际地图对应的分辨率，即仿真地图像素 = 实际距离（m）*（1/0.003）

每块挡片的像素	$0.45 * (1/0.003)$	=150 像素
地图整体长度的像素	$4.512 * (1/0.003)$	=1504 像素
地图整体宽度的像素	$3.162 * (1/0.003)$	=1054 像素

② 自定义地图绘制（以 win10 系统为例）

WindowsPC 下打开画图软件：



图 5-9 画图应用

点击左上角“文件”，选择“属性”选项。在弹出“映像属性”窗口中，勾选单位“像素”，宽度/高度用于设置画布的大小，需要绘制的仿真地图大小为 1504 像素 * 1054 像素，这里取画布的大小为 1510 像素 * 1060 像素。

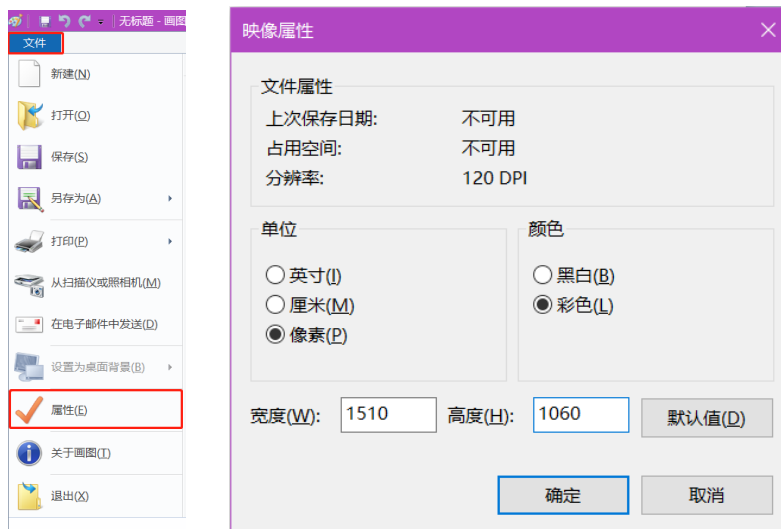


图 5-10 属性面板

完成后点击“确定”，点击左上角“查看”，在弹出的选项中勾选上“标尺”和“网格线”。



图 5-11 查看面板

在画布上。按住 **Ctrl**+鼠标滚轮即可对画布进行放大与缩小。最大效果下，一格即为一个像素点，标尺位于画布的上方和左侧，用于绘画时确认长度。

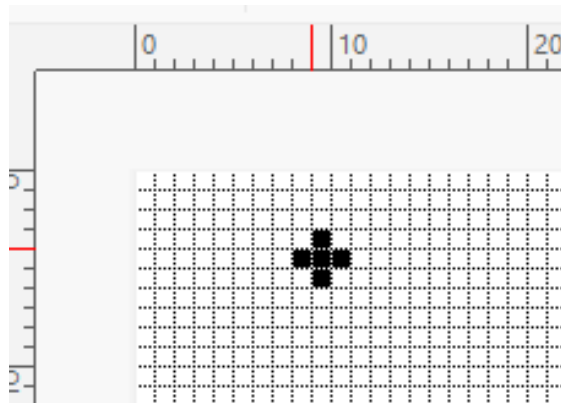


图 5-12 像素格显示

根据需要选择对应的线条形状和粗细，选择粗细时，在格子中绘制线条即可知道线条的宽度，最细的线条为 1 像素。线条颜色使用黑色。

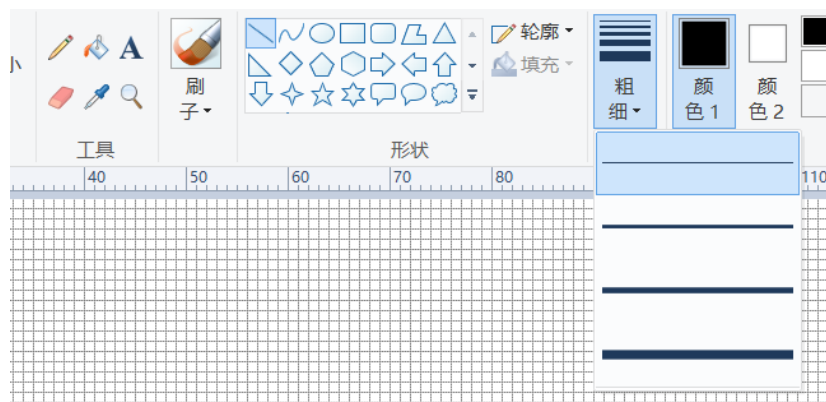


图 5-13 设置画笔

根据实际地图换算的到的仿真地图大小像素数据，即可进行仿真地图绘制。黑色像素即为障碍或墙壁，墙壁线条可酌情增粗。

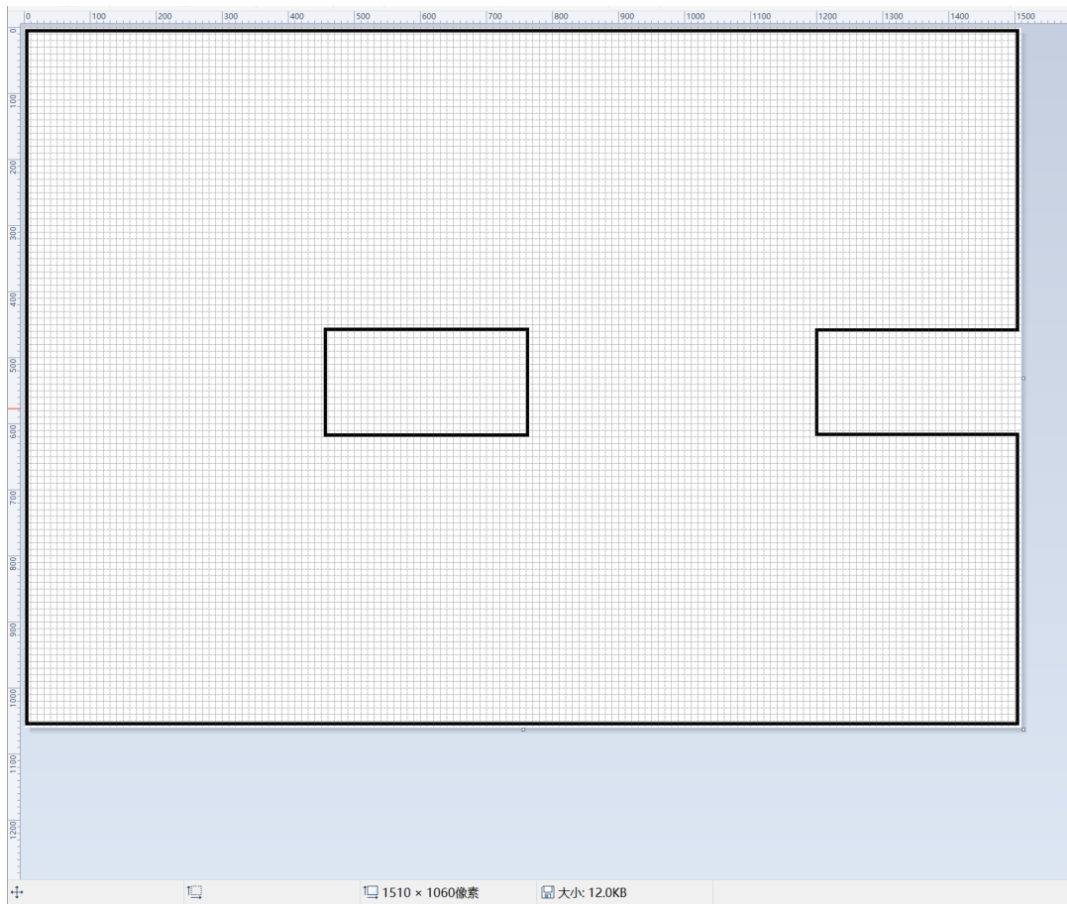


图 5-14 自定义地图示例

绘制完成后进行保存，格式为.PNG 文件。保存完成后将图片文件复制到智能机器人内工程文件夹中的地图目录文件夹 **sebot_simulator/resources/map**。



图 5-15 输出图片

③ 编辑配置文件

将地图文件夹下的任意.yaml 文件复印一份，并将其名称更改为自定义仿真地图的文件名。进入 yaml 文件，对参数进行修改，修改前三项即可。详细参数含义如下。

```
src > sebot_simulator > resources > map > ! sebot_map.yaml
```

1	<code>image: sebot_map.png</code>	#地图文件名称
2	<code>resolution: 0.003</code>	#地图分辨率: 米/像素
3	<code>origin: [0.5, 0.5, 0.0]</code>	#机器人初始位置
4	<code>occupied_thresh: 0.6</code>	#障碍物阈值
5	<code>free_thresh: 0.3</code>	#空闲区域阈值
6	<code>negate: 0</code>	#是否黑白反转

图 5-16 地图配置文件参数

根据需要修改完成后保存配置文件,将最终的地图图片以及配置文件放置机器人内部文件夹 `sebot_simulator/resources/map` 中。

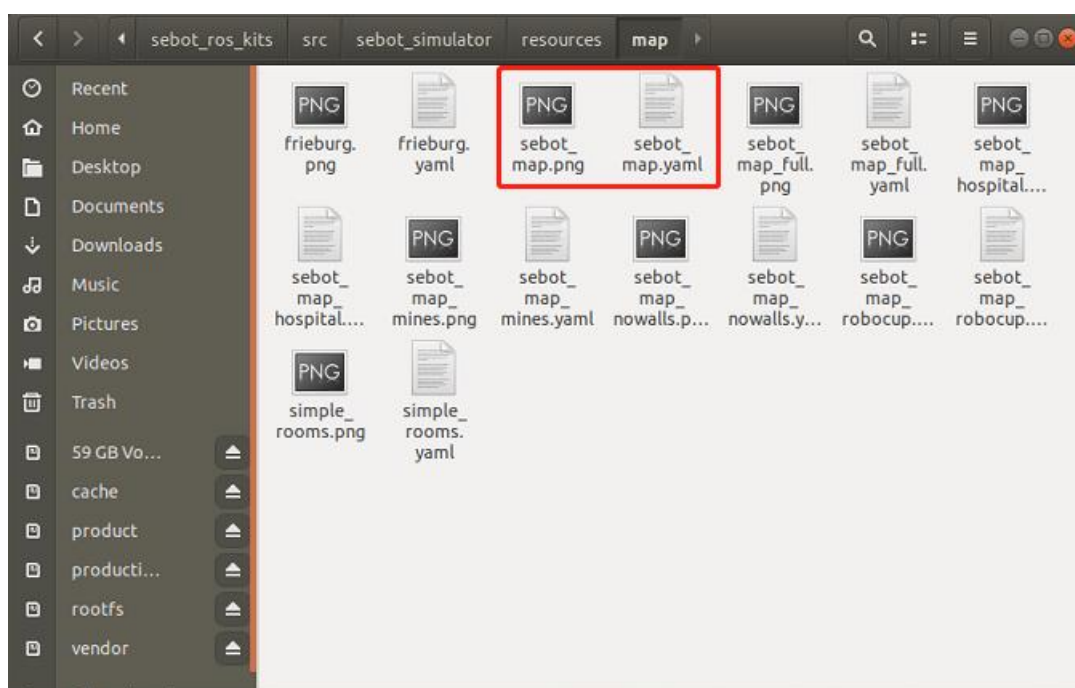


图 5-17 地图图片与配置文件示例

地图自定义完成后,按照前面的步骤修改启动文件 `sebot_std_gui.launch` 中的文件名,然后在终端运行启动文件即可。

需要注意一点,自定义地图仿真时尽量不要绘制过大的地图。加载过大的地图时,仿真准确度以及显示效果都会受到影响。实际地图的尺寸不应超过 20 X 20 (单位:米),具体的地图图片像素值以及参数文件中地图分辨率的设置皆参考此标准。

步骤 2: 仿真定位程序配置逻辑及实操

前一次实训中我们理解了定位算法的参数配置,此步骤中我们将在仿真中观察定位的实操效果。

(1) 配置逻辑

AMCL（自适应蒙特卡洛定位）算法是一种概率学的定位方法，通过对多个例子进行循环计算，最终得出目标在空间中的位置。AMCL 仿真定位的启动文件如下。

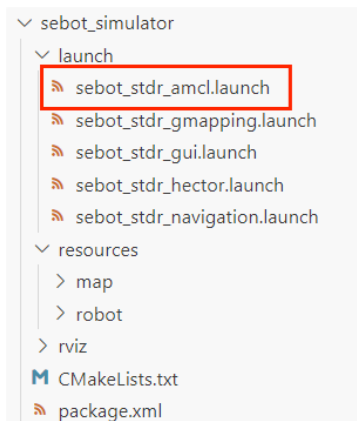


图 5-18 仿真定位启动文件

文件中的配置逻辑如下：

- ① 开启 STDR 仿真器，最终效果即我们前面配置好的仿真器 GUI 界面；
- ② 启动 RViz 监视器，此步骤用于可视化定位效果；
- ③ 加载仿真地图，此步骤是在 RViz 中加载指定地图；
- ④ 启动定位算法，此步骤配置了定位算法的相关参数，具体的配置逻辑和相关文件在前一次实训中已经说明，这里直接调用；
- ⑤ 启动手柄遥控，此步骤是为了开启手柄控制功能，从而实现在 STDR 仿真器和 RViz 监视器中控制机器人模型移动。

详细的配置内容如下。

```
<!-- 基于自适应蒙特卡洛定位 (AMCL) 仿真 -->
<launch>

  <!-- 开启 stdr 仿真器 -->
  <include file = "$(find
sebot_simulator)/launch/sebot_std_r_gui.launch" />

  <!-- 启动 RVIZ 监视器 -->
  <node name = "rviz" pkg = "rviz" type = "rviz" args = "-d $(find
sebot_simulator)/rviz/amcl.rviz" respawn="true"/>

  <!-- 加载仿真地图 -->
  <node pkg="map_server" type="map_server" name="std_r_load_map"
args="$(find sebot_simulator)/resources/map/sebot_map.yaml">
```

```

    <remap from="map" to="/amcl/map" />
</node>

<!-- 启动蒙特卡洛定位算法 -->
<include file="$(find sebot_navigation)/param/amcl.xml">
  <arg name="odom_frame_id" value="map_static"/>
  <arg name="map" value="/amcl/map"/>
  <arg name="init_pose_x" value="1.2"/>      <!-- 机器人初始化姿态: 仿真
与实际差异 -->
  <arg name="init_pose_y" value="1.2"/>      <!-- 机器人初始化姿态: 仿真
与实际差异 -->
  <arg name="init_pose_a" value="1.57"/>      <!-- 机器人初始化姿态: 仿
真与实际差异(角度 rad) -->
</include>

<!-- 启动遥控手柄控制节点 -->
<node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick"
respawn="true"/>

</launch>

```

需要注意的是，STDR 仿真器启动文件中加载的地图和仿真定位启动文件中加载的地图需要保证为同一地图文件，否则可能会出错。

```

src > sebot_simulator > launch > sebot_std_r_amcl.launch
1  <!-- 基于自适应蒙特卡洛定位(AMCL)仿真 -->
2  <launch>
3
4      <!-- 开启stdr仿真器 -->
5      <include file = "$(find sebot_simulator)/launch/sebot_std_r_gui.launch" />
6
7      <!-- 启动RViz监视器 -->
8      <node name = "rviz" pkg = "rviz" type = "rviz" args = "-d $(find sebot_simulator)/rviz/amcl.rviz" respawn="true"/>
9
10     <!-- 加载仿真地图 -->
11     <node pkg="map_server" type="map_server" name="std_r_load_map" args="$(find sebot_simulator)/resources/map/sebot_map.yaml">
12         <remap from="map" to="/amcl/map" />
13     </node>
14
15     <!-- 启动蒙特卡洛定位算法 -->
16     <include file="$(find sebot_navigation)/param/amcl.xml">
17         <arg name="odom_frame_id" value="map_static"/>
18         <arg name="map" value="/amcl/map"/>
19         <arg name="init_pose_x" value="1.2"/>      <!-- 机器人初始化姿态: 仿真与实际差异 -->
20         <arg name="init_pose_y" value="1.2"/>      <!-- 机器人初始化姿态: 仿真与实际差异 -->
21         <arg name="init_pose_a" value="1.57"/>      <!-- 机器人初始化姿态: 仿真与实际差异(角度rad) -->
22     </include>
23
24     <!-- 启动遥控手柄控制节点 -->
25     <node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick" respawn="true"/>
26
27 </launch>

```

图 5-19 确保加载的地图相同

(2) 实操说明

运行启动文件前一定要保证 STDR 仿真器配置文件 `sebot_std_r_gui.launch` 和定位算法仿真配置文件 `sebot_std_r_amcl.launch` 中的加载地图一致。如果前面修改了仿真器加载的地图，可以选择还原仿真器加载的地图或者修改 RViz 监视器加载的地图。

```

src > sebot_simulator > launch > sebot_std_launcher
1 <launch>
2
3 <!-- 启动STDR模拟器 -->
4 <include file="$(find stdr_robot)/launch/robot_manager.launch" />
5
6 <!-- 加载STDR地图 -->
7 <node type="stdr_server_node" pkg="stdr_server" name="stdr_server" output="screen" args="$(find sebot_simulator)/resources/map/sebot_map.yaml"/>
8
9 <!-- 发布地图静态坐标系 -->
10 <node pkg="tf" type="static_transform_publisher" name="world2map" args="0 0 0 0 0 world map 30" />
11
12 <!-- 启动STDR图形化界面 -->
13 <include file="$(find stdr_gui)/launch/stdr_gui.launch"/>
14
15 <!-- 加载服务器机器人物理模型 (x y th) 机器人起始位置 -->
16 <node pkg="stdr_robot" type="robot_handler" name="$(anon robot_spawn)" args="add $(find sebot_simulator)/resources/robot/sebot_robot.yaml 0.5 0.5 1.57" />
17
18 </launch>

```

图 5-20 STDR 仿真器加载的地图

```

src > sebot_simulator > launch > sebot_std_launcher
1 <!-- 基于自适应蒙特卡洛定位(AMCL)仿真 -->
2 <launch>
3
4 <!-- 开启stdr仿真器 -->
5 <include file = "$(find sebot_simulator)/launch/sebot_std_launcher.launch" />
6
7 <!-- 启动RViz监视器 -->
8 <node name = "rviz" pkg = "rviz" type = "rviz" args = "-d $(find sebot_simulator)/rviz/amcl.rviz" respawn="true"/>
9
10 <!-- 加载仿真地图 -->
11 <node pkg="map_server" type="map_server" name="stdr_load_map" args="$(find sebot_simulator)/resources/map/sebot_map.yaml">
12 | <remap from="map" to="/amcl/map" />
13 </node>
14
15 <!-- 启动蒙特卡洛定位算法 -->
16 <include file="$(find sebot_navigation)/param/amcl.xml">
17 | <arg name="odom_frame_id" value="map_static"/>
18 | <arg name="map" value="/amcl/map"/>
19 | <arg name="init_pose_x" value="1.2"/> <!-- 机器人初始化姿态: 仿真与实际差异 -->
20 | <arg name="init_pose_y" value="1.2"/> <!-- 机器人初始化姿态: 仿真与实际差异 -->
21 | <arg name="init_pose_a" value="1.57"/> <!-- 机器人初始化姿态: 仿真与实际差异(角度rad) -->
22 </include>
23
24 <!-- 启动遥控手柄控制节点 -->
25 <node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick" respawn="true"/>
26
27 </launch>

```

图 5-21 仿真定位功能加载的地图

确保一致后在终端中输入以下命令启动定位算法仿真程序。

```
roslaunch sebot_simulator sebot_std_launcher.launch
```

程序运行后，会分别加载 STDR 仿真器和 RViz 监视器，等待二者都加载完成后，可以看到 RViz 中出现很多用于定位计算的粒子，覆盖在 RViz 监视器地图障碍物上的动态彩色线条即激光雷达扫描到的障碍物信息。

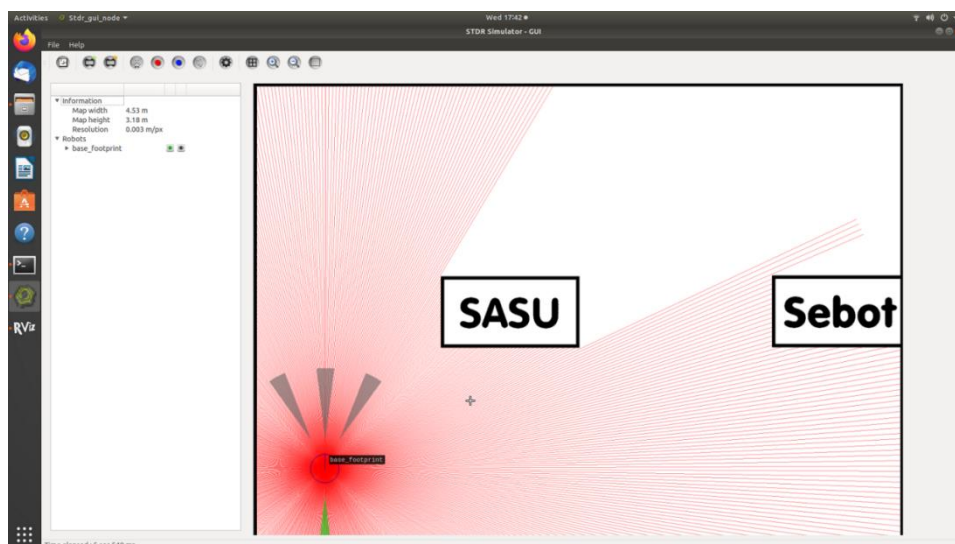


图 5-22 程序启动后的 STDR 仿真器界面

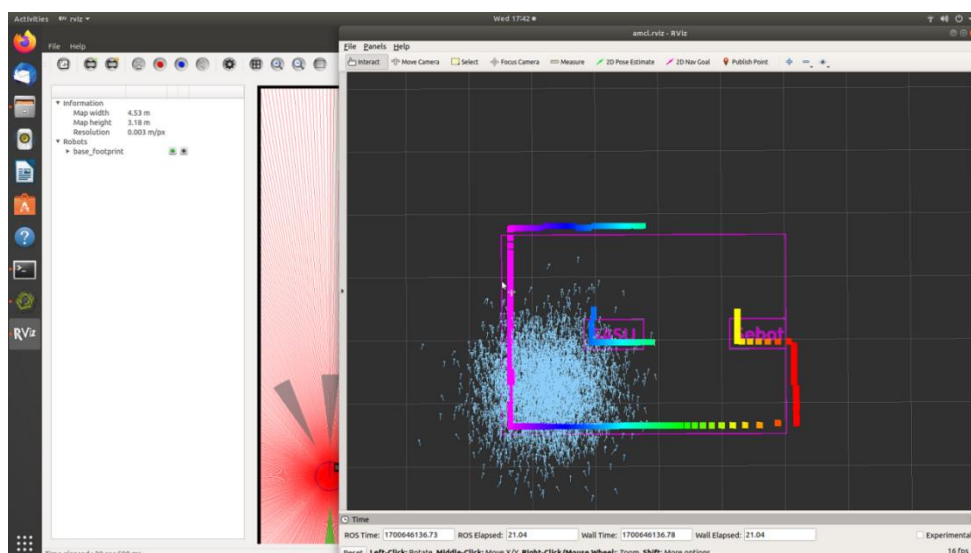


图 5-23 程序启动后的 RViz 监视器界面

通过手柄控制仿真机器人移动，可以观察到 STDR 仿真器中的仿真机器人模型发生移动，同时 RViz 监视器中的粒子发生移动且数量逐渐减少。随着 RViz 监视器中的粒子减少，对仿真机器人的定位越精准。

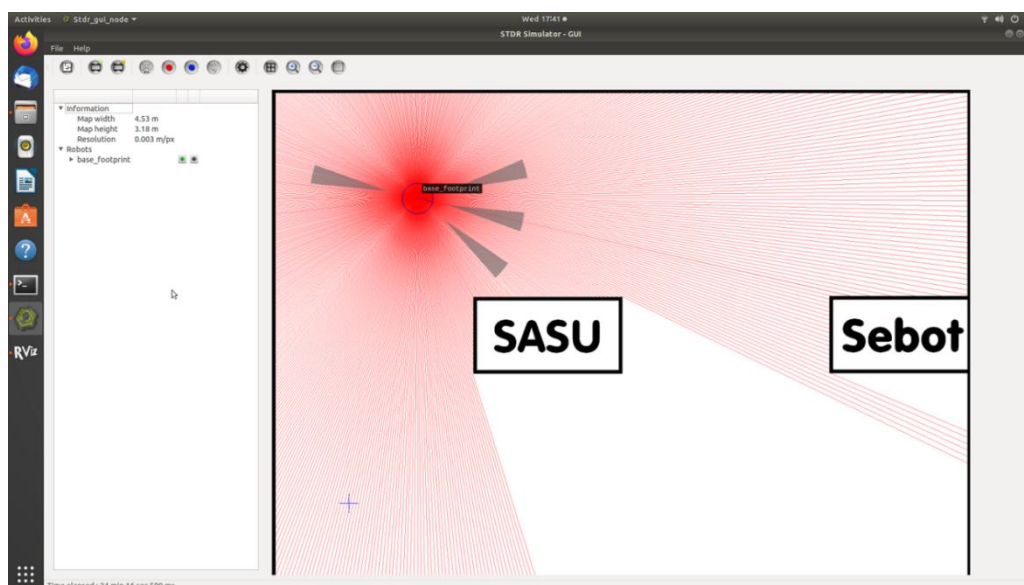


图 5-24 控制仿真机器人移动后的 STDR 仿真器界面

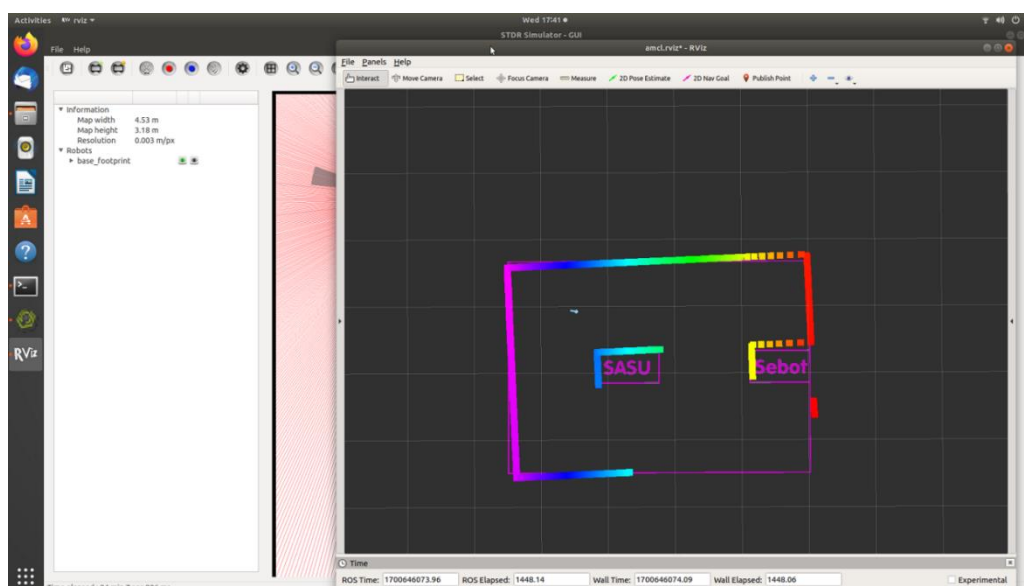


图 5-25 控制仿真机器人移动后的 RViz 监视器界面

步骤 3：仿真建图程序配置逻辑及实操

仿真建图功能基于 STDR 仿真器中加载的地图文件，在仿真器中控制机器人移动时，会发出对应的话题，根据这些话题内的数据，在 RViz 监视器中即可生成对应的地图文件。

(1) 配置逻辑

仿真建图的程序与前面的真实建图一致，也使用了 Gmapping 和 Hector 两种算法，其配置文件如下：

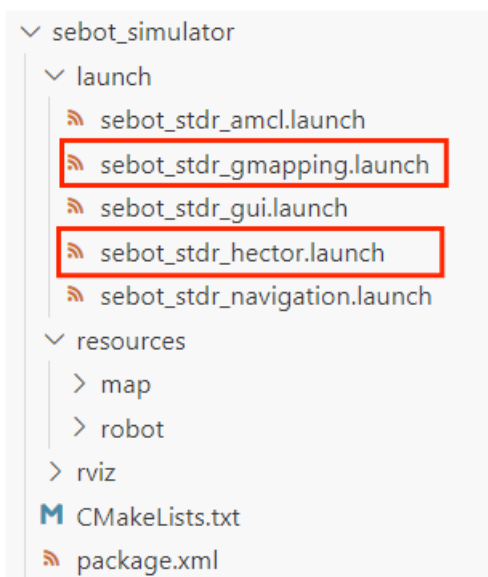


图 5-26 两种算法仿真建图的启动文件

在这两个配置文件中，最大的差别即相关建图算法的参数，加载的算法参数是从真实建图的配置文件（前面建图实训中的学习内容）中获取的。其他的步骤完全一致，即：

- ① 启动 STDR 仿真器，此步骤会加载预设的地图；
- ② 启动 RViz 监视器，此步骤用于获取仿真中得到的数据；
- ③ 启动建图算法，此步骤用于根据获取的数据进行建图操作；
- ④ 启动手柄控制，此步骤用于控制机器人在 STDR 仿真器中移动，从而获取相关数据。

Gmapping 算法仿真建图的配置文件内容如下：

```
<!-- 使用 stdr 的仿真进行 gmapping 建图 -->
<launch>

  <!-- 开启 stdr 仿真器 -->
  <include file = "$(find
sebot_simulator)/launch/sebot_std_r_gui.launch" />

  <!-- 启动 RVIZ 监视器 -->
  <node name = "rviz" pkg = "rviz" type = "rviz" args = "-d $(find
sebot_simulator)/rviz/slam.rviz" respawn="true"/>

  <!-- 启动 gmapping 建图算法 -->
  <include file="$(find sebot_slam)/param/gmapping.xml">
    <arg name="map_topic" value="/slam_map" /> <!-- 发布的地图话题名称
(map_topic),防止和 stdr 发布的地图名称重叠 -->
    <arg name="odom_frame" value="map_static" />
  </include>
```



```

<!-- 启动遥控手柄控制节点 -->
<node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick"
respawn="true"/>

</launch>

```

Hector 算法仿真建图的配置文件内容如下：

```

<!-- 使用 stdr 的仿真进行 hector 建图 -->
<launch>

  <!-- 开启 stdr 仿真器 -->
  <include file = "$(find
sebot_simulator)/launch/sebot_std_gui.launch" />

  <!-- 启动 RVIZ 监视器 -->
  <node name = "rviz" pkg = "rviz" type = "rviz" args = "-d $(find
sebot_simulator)/rviz/slam.rviz" respawn="true"/>

  <!-- 开启 hector 建图算法节点 -->
  <include file="$(find sebot_slam)/param/hector.xml">
    <arg name="map_topic" value="/slam_map" />          <!-- 发布的
地图话题 -->
    <arg name="use_tf_pose_start_estimate" value="true"/>  <!-- 启动
姿态估计 -->
  </include>

  <!-- 启动遥控手柄控制节点 -->
  <node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick"
respawn="true"/>

</launch>

```

(2) 实操说明

根据需求和前面的参考步骤自行修改 STDR 仿真器启动文件 `sebot_std_gui.launch` 中加载的地图文件，保存后即可在终端中输入对应命令启动对应算法的仿真建图。

Gmapping 算法仿真建图程序命令如下：

```
roslaunch sebot_simulator sebot_std_gui_gmapping.launch
```

Hector 算法仿真建图程序命令如下：

```
roslaunch sebot_simulator sebot_std_gui_hector.launch
```

两种算法的仿真建图程序启动后的界面一致，在启动对应的程序后，系统会启动 STDR

仿真器和 RViz 监视器，STDR 仿真器内是加载好的地图，RViz 监视器内是要建立的地图，通过手柄控制仿真机器人在 STDR 仿真器的地图内移动(实物机器人在真实环境中不移动)，然后观察 RViz 监视器内的建图结果即可。

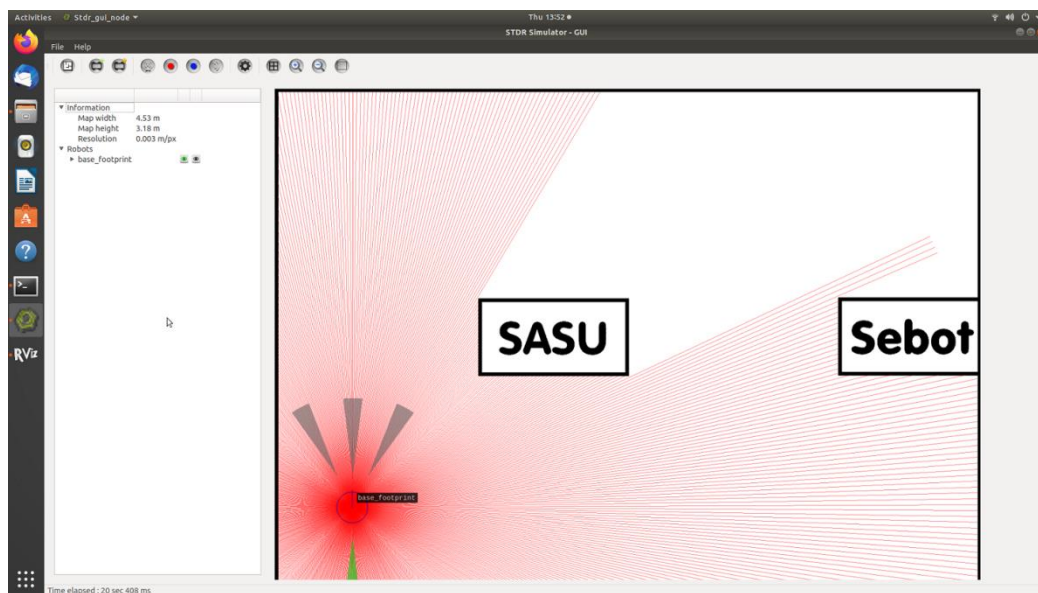


图 5-27 程序启动后的 STDR 仿真器

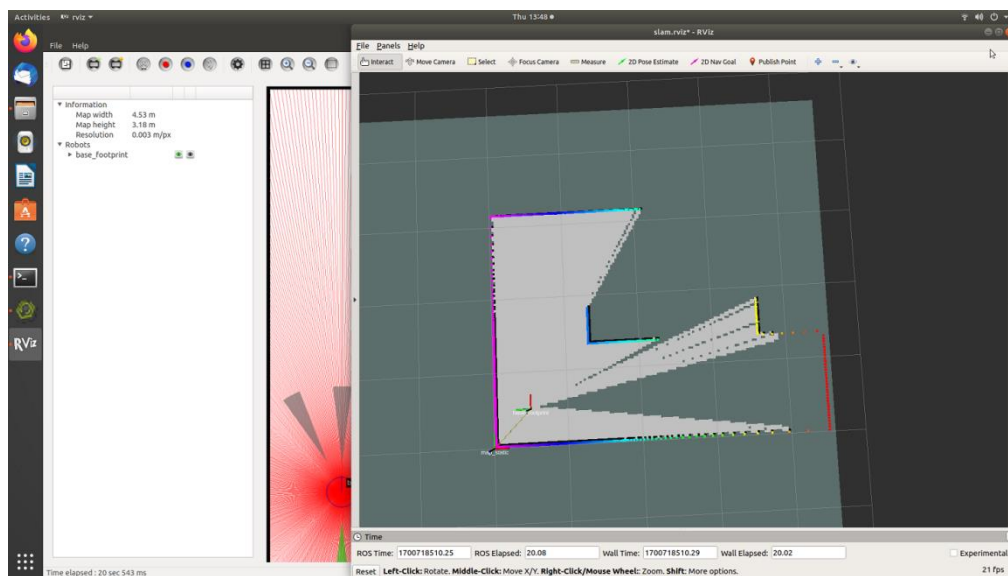


图 5-28 程序启动后的 RViz 监视器

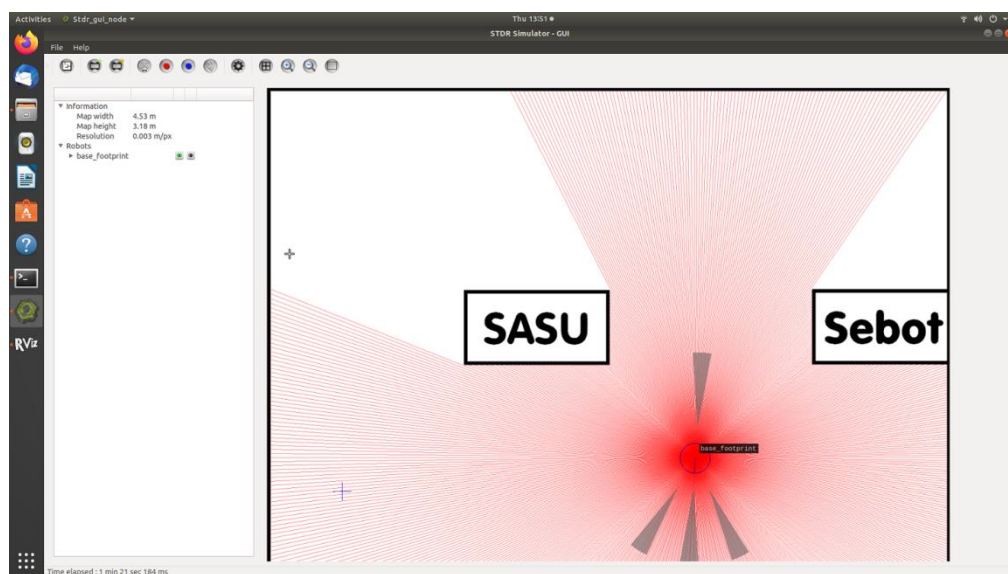


图 5-29 绕图移动一圈后的 STDR 仿真器

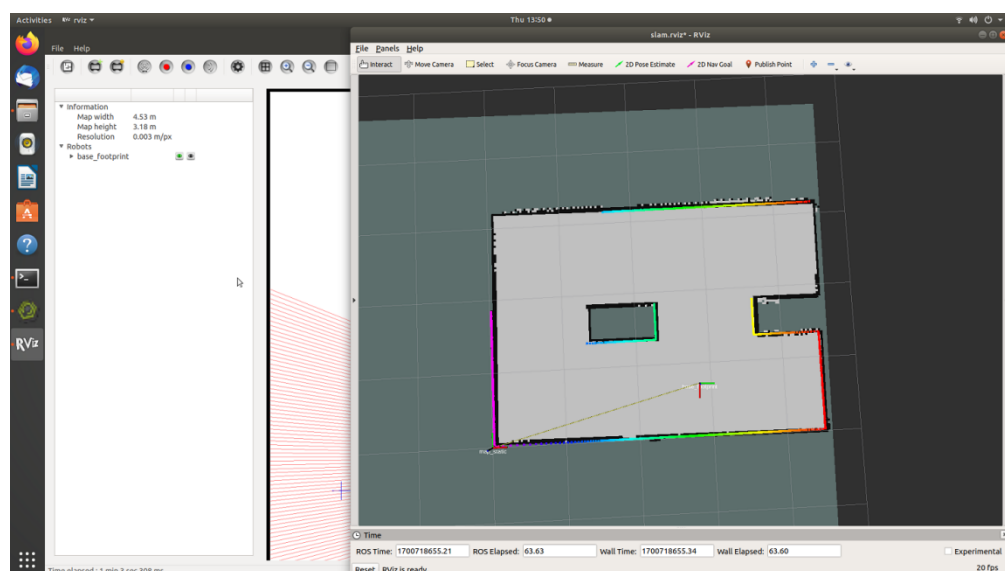


图 5-30 绕图移动一圈后的 RViz 监视器

如果需要保存地图进行分析，则打开新终端，先使用 `cd` 命令进入到想要保存地图文件的目录，然后输入以下保存指令，地图文件名可自行设置。保存成功后将加载到仿真器中的地图图片和建图完成后保存的地图图片对比形状即可验证建图结果。

```
rosrun map_server map_saver map:=/slam_map -f 地图文件名
```

```
root@paddlepi:~# cd workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_simulator/resources/map/
root@paddlepi:~/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_simulator/resources/map# roslaunch map_server map_saver map:=/slam_map -f map1
```

图 5-31 保存地图操作

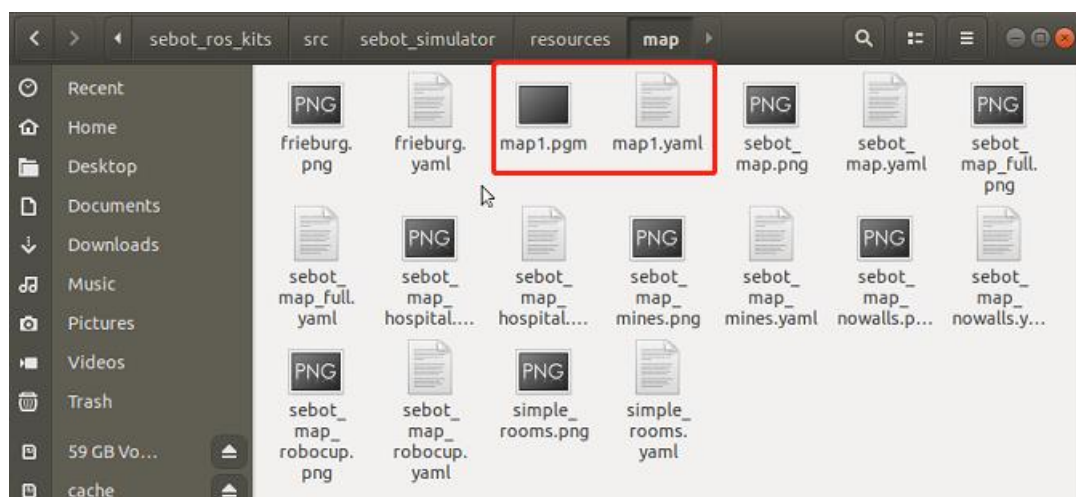


图 5-32 保存的地图文件

步骤 4：仿真导航程序配置逻辑及实操

智能机器人的仿真导航功能是基于加载的地图文件，使用的参数是调用的真实环境中导航功能的参数，最终效果是在 RViz 监视器中控制仿真机器人进行路径规划并移动。

（1）配置逻辑

仿真导航的启动文件如下：

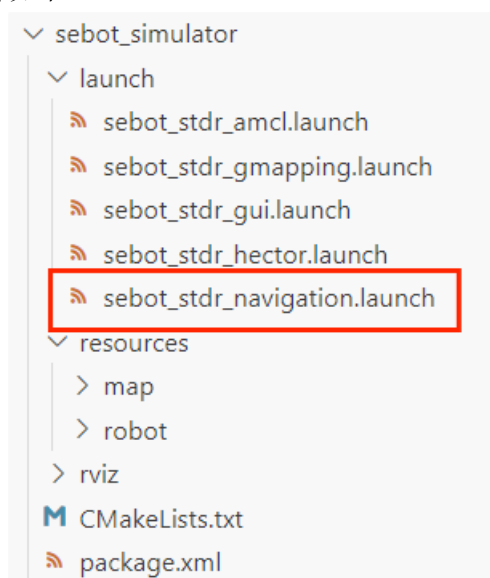


图 5-33 仿真导航启动文件

文件中的配置逻辑为：

- ① 开启 STDR 仿真器；
- ② 启动 move_base 节点，此节点用于机器人进行路径规划和移动控制，加载的相关参数即前面实训中学习的真实环境中导航的配置文件的参数；
- ③ 启动 RViz 监视器，此步骤用于可视化运行结果；
- ④ 加载仿真地图，机器人在 RViz 监视器中进行导航功能的地图即此步骤设置的地图

文件；

- ⑤ 启动蒙特卡洛定位算法，此步骤是用于在导航功能中实现机器人的自主定位功能，加载的相关参数即前面实训中学习的真实环境中定位的配置文件的参数。

详细的配置内容如下：

```
<!-- stdr 地图导航 -->
<launch>

  <!-- 开启 stdr 仿真器 -->
  <include file = "$(find
sebot_simulator)/launch/sebot_std_gui.launch" />

  <!-- 启动 move_base 节点 -->
  <include file = "$(find sebot_navigation)/param/move_base.xml">
    <arg name="odom_frame_id" default="map_static"/>
    <arg name="map_topic" default="/amcl/map"/>
  </include>

  <!-- 启动 RVIZ 监视器 -->
  <node name = "rviz" pkg = "rviz" type = "rviz" args = "-d $(find
sebot_simulator)/rviz/navigation.rviz" respawn="true"/>

  <!-- 加载仿真地图 -->
  <node pkg="map_server" type="map_server" name="std_load_map"
args="$(find sebot_simulator)/resources/map/sebot_map.yaml"
respawn="true">
    <remap from="map" to="/amcl/map" />
  </node>

  <!-- 启动蒙特卡洛定位算法 -->
  <include file="$(find sebot_navigation)/param/amcl.xml">
    <arg name="odom_frame_id" value="map_static"/>
    <arg name="map" value="/amcl/map"/>
    <arg name="init_pose_x" value="1.0"/>          <!-- 机器人初始化姿态：仿真
与实际差异 -->
    <arg name="init_pose_y" value="1.0"/>          <!-- 机器人初始化姿态：仿真
与实际差异 -->
    <arg name="init_pose_a" value="1.57"/>          <!-- 机器人初始化姿态：仿
真与实际差异(角度 rad) -->
  </include>

</launch>
```

(2) 实操说明

与仿真建图功能相似，在实操前需要先确保 STDR 仿真器和 RViz 监视器中加载的地图文件一致。如果修改了地图文件，注意在 25~27 行修改机器人的初始姿态，确保机器人处于地图内部，如果机器人处于地图外部，则无法进行路径规划。

```
src> sebot_simulator > launch > sebot_stdnav.launch
1 <!-- stdr地图导航 -->
2 <launch>
3
4 <!-- 开启stdr仿真器 -->
5 <include file = "$(find sebot_simulator)/launch/sebot_stdnav_gui.launch" />
6
7 <!-- 启动move_base节点 -->
8 <include file = "$(find sebot_navigation)/param/move_base.xml">
9   <arg name="odom_frame_id" default="map_static"/>
10  <arg name="map_topic" default="/amcl/map"/>
11 </include>
12
13 <!-- 启动RVIZ监视器 -->
14 <node name = "rviz" pkg = "rviz" type = "rviz" args = "-d $(find sebot_simulator)/rviz/navigation.rviz" respawn="true"/>
15
16 <!-- 加载仿真地图 -->
17 <node pkg="map_server" type="map_server" name="stdr_load_map" args="$(find sebot_simulator)/resources/map/sebot_map.yaml" respawn="true">
18   <remap from="map" to="/amcl/map" />
19 </node>
20
21 <!-- 启动蒙特卡洛定位算法 -->
22 <include file="$(find sebot_navigation)/param/amcl.xml">
23   <arg name="odom_frame_id" value="map_static"/>
24   <arg name="map" value="/amcl/map"/>
25   <arg name="init_pose_x" value="1.0"/> <!-- 机器人初始化姿态: 仿真与实际差异 -->
26   <arg name="init_pose_y" value="1.0"/> <!-- 机器人初始化姿态: 仿真与实际差异 -->
27   <arg name="init_pose_a" value="1.57"/> <!-- 机器人初始化姿态: 仿真与实际差异(角度rad) -->
28 </include>
29
30 </launch>
```

图 5-34 确认仿真器和监视器

文件修改完成并保存后在终端输入以下指令即可开启仿真导航功能。

```
roslaunch sebot_simulator sebot_stdnav_navigation.launch
```

程序启动后会自动打开 STDR 仿真器和 RViz 监视器，在 RViz 监视器中设置终点后即可观察到仿真机器人自行导航（起点已经通过配置文件设置完成），操作步骤与真实环境中导航的操作一致，区别是仿真环境下，实物机器人不发生真实移动。

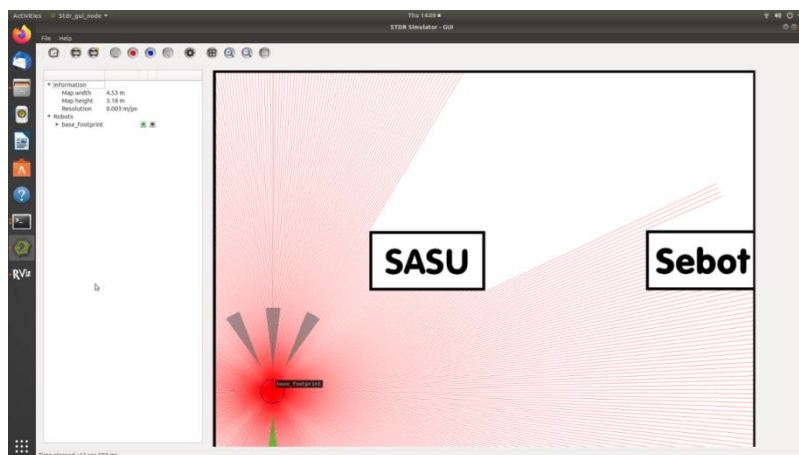


图 5-35 程序启动后的 STDR 仿真器

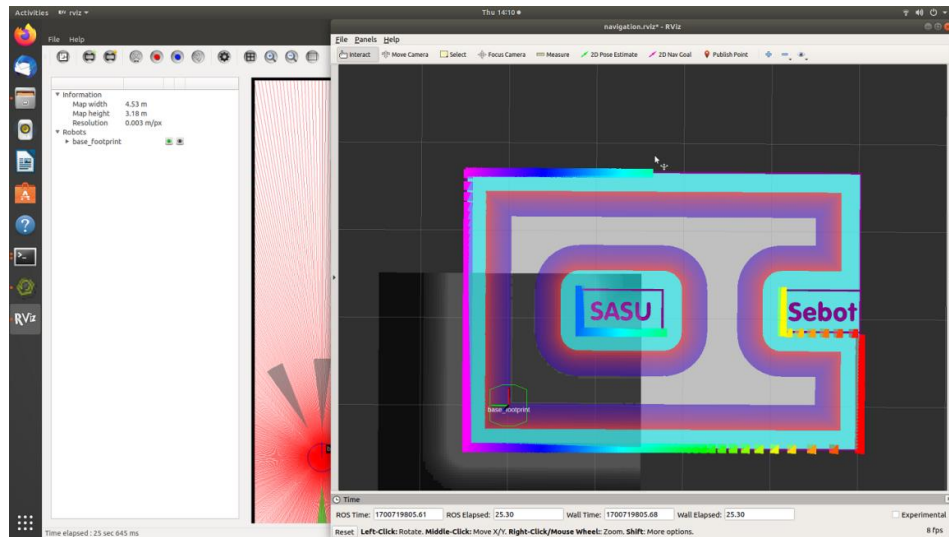


图 5-36 程序启动后的 RViz 监视器

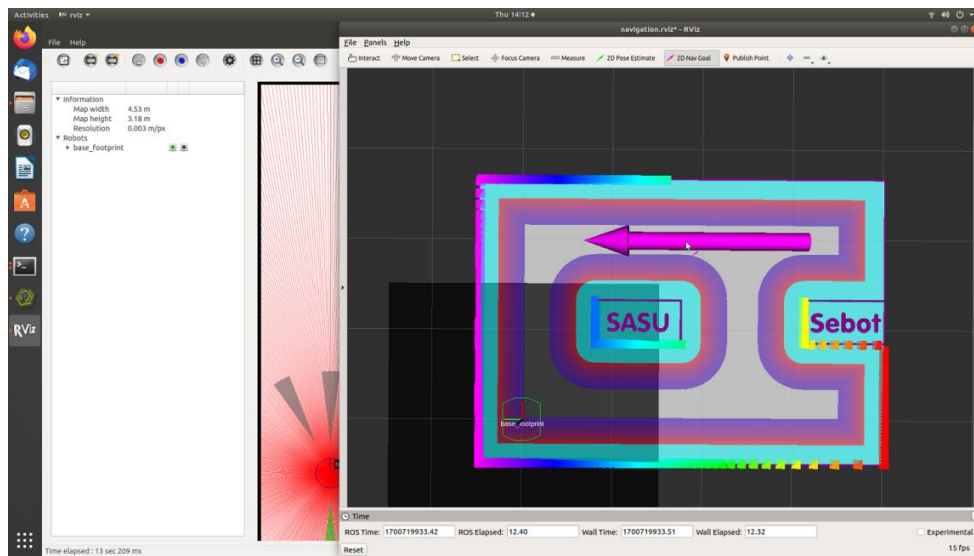


图 5-37 设置终点位置

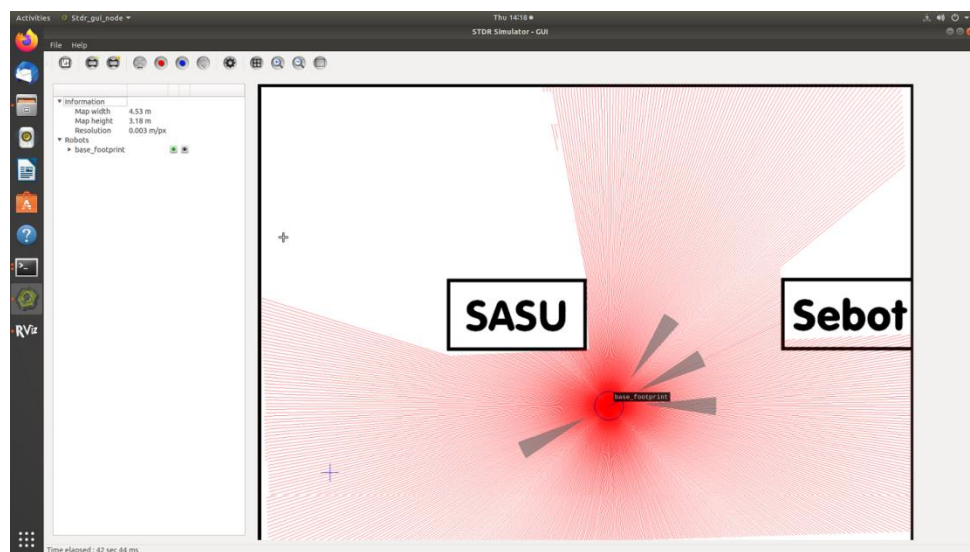


图 5-38 STDR 仿真器中机器人导航移动中

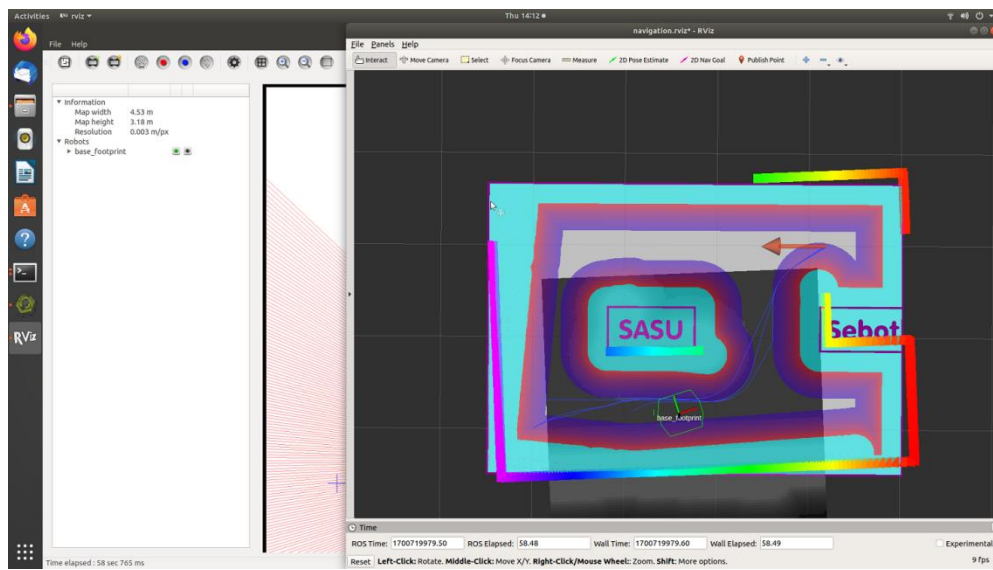


图 5-39 RViz 监视器中机器人导航移动中

六、语音交互实验指导

1. 实验描述

智能机器人在酒店楼层工作的时候，可能会遇到行人、酒店工作人员，或是客房里的客人，通过语音等自然语言交互能显著提升人们的服务体验，此时如果客户有需求，例如机器人行进时，路过客房，客房的客人与机器人打招呼并且表示“需要一瓶水”，机器人便需要从嘈杂的环境下提取对方的语音，辨别用户的语音需求，并且同步做出相应的语音反馈或动作反馈。

本次课我们尝试完成语音交互的所有功能，把用户输入的语音转换成文字，针对文字内容进行反馈，将反馈的内容生成音频传达给用户，或通过语音控制机器人开启某功能，完成语音交互任务。

2. 实验目标

- （1）掌握语音交互的逻辑与技术实现方法；
- （2）熟悉语音交互的相关应用技术，掌握语音交互的源码编程逻辑；
- （3）会操作智能机器人进行语音交互的完整实践，会对相关步骤进行调试。

3. 语音交互的流程

如下图所示为本实验语音交互的逻辑流程图。在本课程实验中，需要注意的是与智能机器人的语音互动中需要保证机器人处于联网状态，机器人没有设置语音唤醒功能，需要通过按键“空格”开启录音的方式读取声音，从而完成语音交互的整个流程。

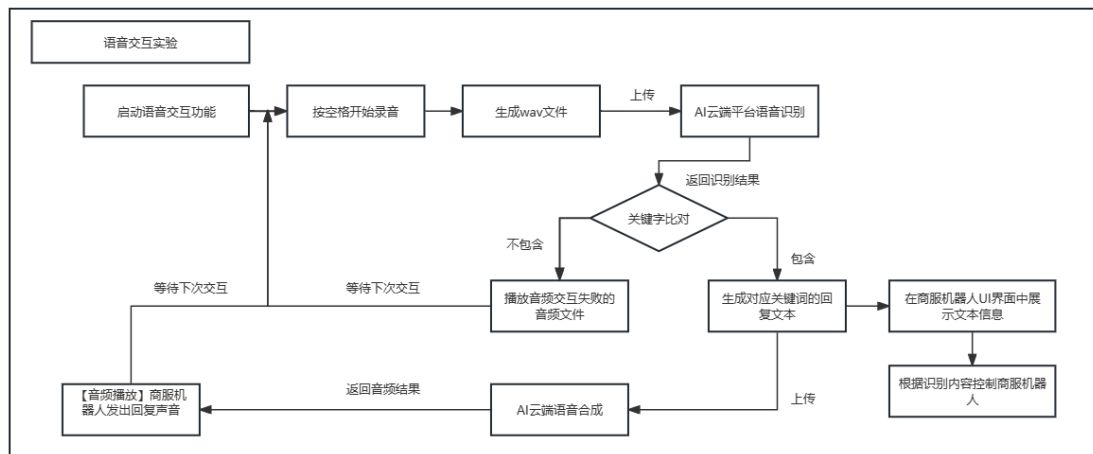


图 6-1 语音交互逻辑

第一步,录音采集。对传入的声音进行录音,为了方便后续提高语音识别的速度和效率,会提前创建完成一个音频文件,文件名称为“temp.wav”,将录音采集到的声音数据存储到该文件中,伴随着语音的多次输入,该文件每次都会被新输入的语音覆盖和刷新;

第二步,语音识别。本次语音交互的过程结合百度智能云平台去完成,将本地生成的“temp.wav”文件的音频内容上传至百度智能云平台,智能云平台会进行语音识别,然后返回与之对应的文本内容到本地,此步骤则完成语音识别的任务。

第三步,关键词匹配。关键词匹配是智能语音交互中应用的重要技术之一,涉及到回复内容的产生。智能机器人在本地专门设立的关键词匹配文件,里面存储了一些关键词所对应的回复文本内容。如果识别到的语音内容中含有回复信息,则进行下一步的语音合成;如果识别到的语音内容中不含有回复信息,则直接播放本地的音频文件,播报“抱歉,我没有明白您说的话,您可以再讲一次!”等音频。

第四步,语音控制。语音交互不仅要包含人与机器人之间的语音沟通,还需要包含人与人之间的语音控制。语音交互功能启动后会生成 UI 界面,如下图所示,一共分为四大功能: control、detect、enjoy、action 这四项功能,可以根据界面中的文字提示输入语音内容,根据前一个步骤得到的语音识别后的文本信息,就可以进一步判断理解文本的需求,得到相应的回复内容在 UI 界面中进行展示,同时进行智能机器人控制。例如当智能机器人 UI 功能界面启动后,按下空格输入语音“智能机器人向前移动”、“开启自主导航”、“播放音乐”、“当前日期时间”等语音内容,智能机器人就会反馈相应的行为或语音。

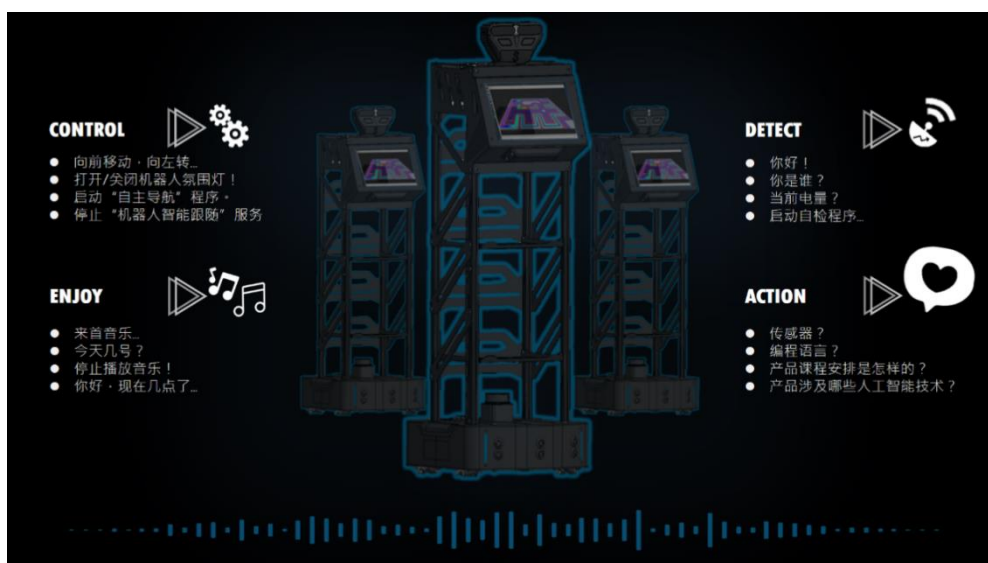


图 6-2 语音交互 UI 界面

第五步，语音合成。在第三步中得到了对应的回复文本，继续将该文本上传至百度智能云平台中，调用百度智能云的语音合成 API 功能，完成语音合成的任务，进而将合成的音频文件（回答内容）返回至智能机器人本地，并通过扬声器播放出来，这样我们就可以在智能机器人中得到自己想要的回复内容，具体查看“interaction.json”文本文件，后续也可以自己在该文件中自定义语音回复的内容。

一轮语音交互过程完成后，再次按下空格按键，重复上述步骤，就可以开启继续新一轮的语音交互进程。与此同时，还有键盘相关的控制操作，在后续实操过程中进行展示。根据如上的语音交互逻辑，有如下实验步骤以及代码逻辑内容讲解。

4. 实验步骤

步骤 1：创建百度智能云语音识别应用

本次实验需要调用百度智能云的语音识别以及语音合成功能，我们需要借助调用云端 API 的方式完成语音交互中的两项关键功能。

首先是注册账号，登录百度智能云的官网，官网链接“<https://cloud.baidu.com/>”，注册个人账号并进行认证后，点击控制台即可选择具体的产品服务。



图 6-3 百度智能云官网

进入控制台后选择左上角的按钮，然后选择产品服务，在人工智能分类中选择语音技术即可。

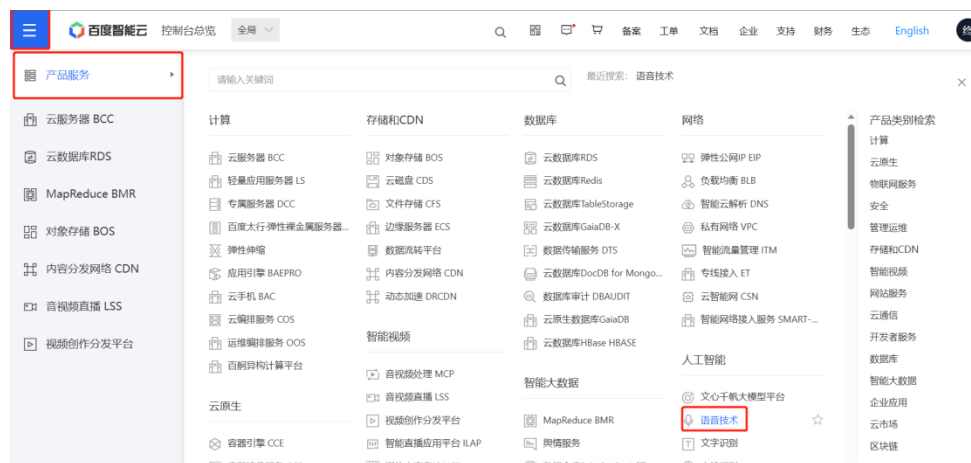


图 6-4 选择语音技术

进入语音技术应用后，根据智能云平台的概览引导，首次注册的用户可以免费领取云端语音技术的资源，包括语音识别、语音合成与呼叫中心语音这三个部分。

需要注意的是，要切换到每个模块下领取对应模块的资源，不要忘记领取，否则后续申请接口的语音资源时，则会失去部分的功能。

领取所有资源后，在左侧选择应用列表，然后创建应用项目：



图 6-5 创建应用

创建应用后需要选择服务类型并填写相关信息，根据实际情况填写即可，因为我们后续实验过程需要完成语音识别与语音合成两项任务，所以勾选“全选”将所有功能选上。

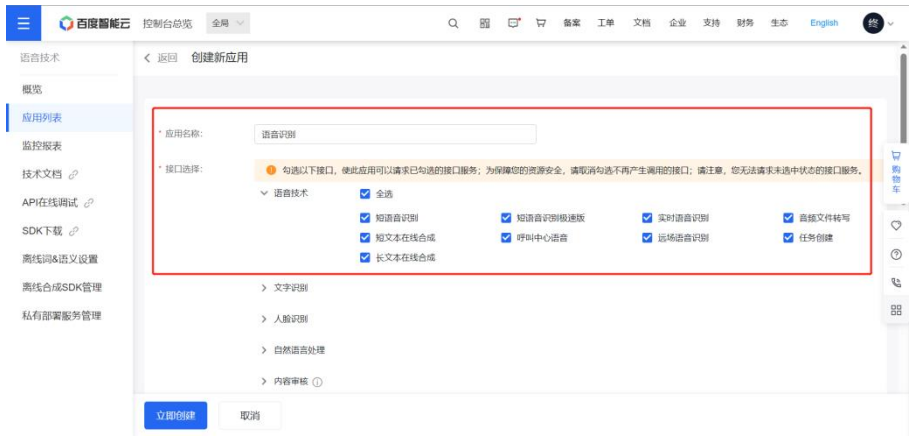


图 6-6 选择服务



图 6-7 填写信息

填写完成后选择立即创建，此时从应用列表中即可查看我们创建的应用“语音识别”：

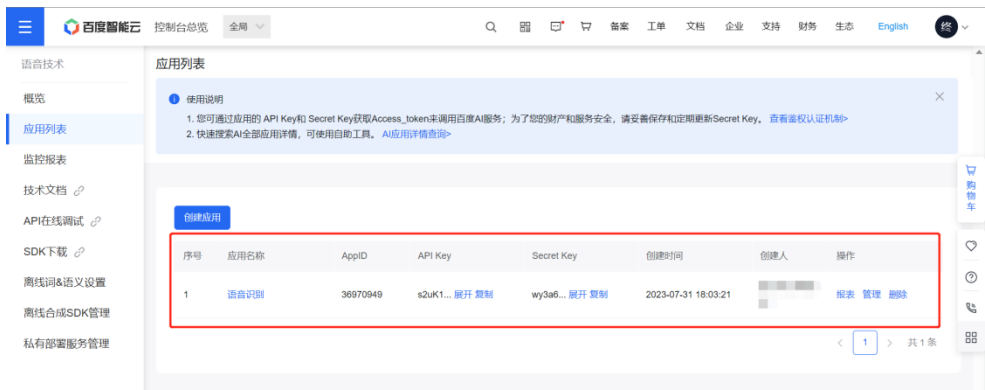


图 6-8 已创建的应用

创建应用后我们需要记录应用信息中的 API Key 和 Secret Key，这两个信息是后续编程中需要使用到的重要内容。因为只有通过这两项关键密钥，才可以请求云端 API 接口中的语音识别以及语音合成功能。



图 6-9 API Key 和 Secret Key

步骤 2：认识语音交互的工程文件

参考实训 1 的步骤,通过连接网线、以太网设置等方式将个人 PC 远程连接智能机器人，在个人 PC 中使用 VSCode 通过远程连接的方式打开智能机器人中的工程文件 sebot_t710_workspace，在路径 “/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/scripts/aiInteraction.py” 找到本实验语音交互的 python 工程代码文件 aiInteraction.py。

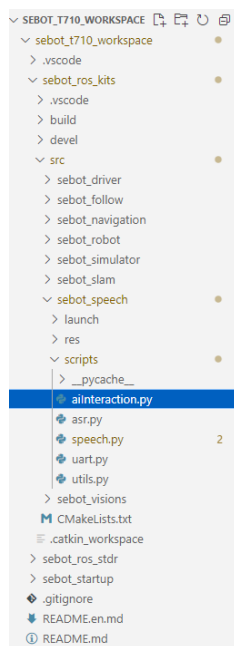


图 6-10 文件夹所在路径

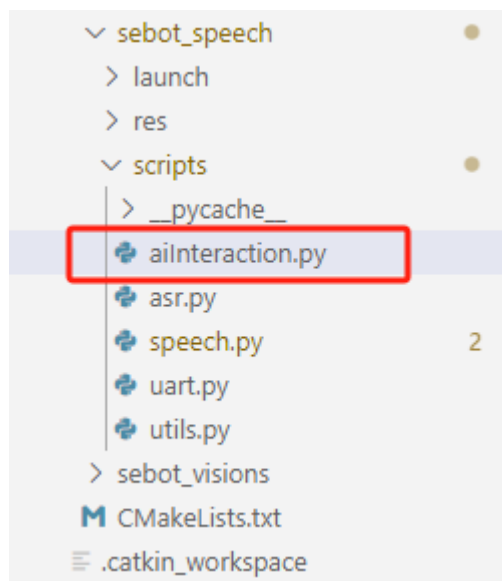


图 6-11 语音交互脚本代码所在路径

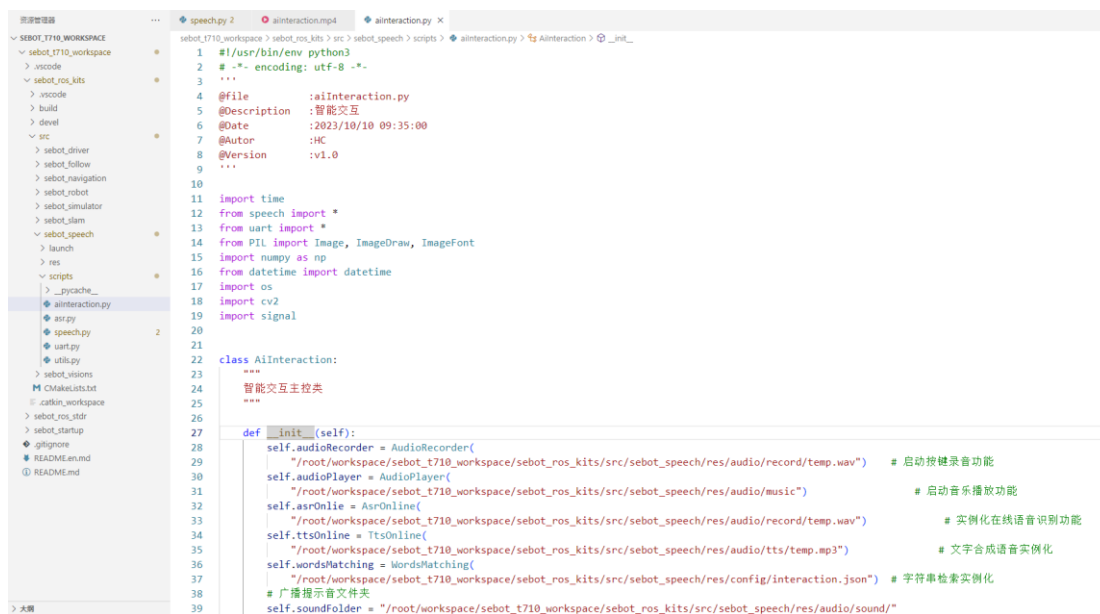


图 6-12 对应的语音交互代码文件界面

根据前文所讲解的语音交互逻辑，语音交互系统中的核心代码一共分为两个部分，一个是同级目录下的 `speech.py` 文件，包含语音识别、音频播放器、关键词匹配以及语音合成这几项功能；另外一个为主程序文件 `aiInteraction.py` 文件，包含有界面启动、文本内容判断、控制语音输出、显示 UI 界面内容等控制，主要负责调用其他部分的功能函数进行集成实现完整的语音合成系统。

针对于这两部分的功能代码，接下来我们分开进行详细的代码逻辑内容讲解。首先讲解 `speech.py` 文件中的代码逻辑，之后讲解主函数程序 `aiInteraction.py` 中的代码逻辑。

步骤 3：理解语音技术相关功能的代码逻辑（speech.py）

语音交互程序文件 `speech.py` 对应有如下几部分主要功能：① 导入库函数及定义 API 密钥；② 录音功能类，用于录音功能，并且可以将录制的音频存放至本地；③ 播放器类，包含有完整的音乐播放器功能，但是在本实训中主要用到该类中的部分函数功能，例如将本地的音频进行播放；④ 语音识别类，用于将本地的录音生成的音频文件上传至百度智能云平台完成语音识别任务；⑤ 关键词匹配与内容生成类，用于根据语音识别后的文本进行内容生成；⑥ 语音合成类，将关键词匹配后得到的回复内容发送至百度智能云平台进行音频生成，完成语音合成的任务。（由于代码篇幅过长，完整代码请参考代码源文件进行查看）

1、导入库函数（scripts/speech.py）

- （1）导入 `pyaudio` 库：该库用于进行录音，播放，生成 wav 语音文件等操作。
- （2）导入 `wave` 库：该库的作用是操作 wav 格式的文件。
- （3）导入 `keyboard` 库：该库的作用是实现程序中的键盘交互功能。
- （4）导入 `os` 库：操作系统相关功能,比如路径操作。
- （5）导入 `random` 库:生成随机数。
- （6）导入 `pygame` 库：用于提示音或语音合成结果。
- （7）导入 `urllib` 库的四个函数，从 `urllib` 导入 `request, error, parse`:处理网络请求,发送请求到百度语音 API,也处理请求错误。`urlencode` 编码参数,`quote_plus` 处理 URL。
- （8）导入 `json` 库。该库主要用于将 `python` 对象编码为 `json` 格式输出或存储，以及将 `json` 格式对象解码为 `python` 对象。
- （9）导入 `base64` 库。该库提供了大量函数用来把二进制数据编码为可打印的 `ASCII` 字符，以及将其解码为二进制数据。
- （10）导入 `func_timeout` 库。设置函数执行超时,可能用于识别请求的超时控制。
- （11）导入 `utils` 库。引入自己定义的一些工具函数。

剩下两行定义密钥的代码则是为了在调用 API 时可以方便使用，在前文中的智能云平台中获得这两个密钥就可以在此处对应的位置中进行替换。（注意：本实训在文档中不提供密钥的具体信息，需要自行从官网注册得到密钥）

```
import pyaudio
import wave
import keyboard
import os
import random
```

```
import pygame
from pygame.locals import *
from urllib.request import urlopen
from urllib.request import Request
from urllib.error import URLError
from urllib.parse import urlencode
from urllib.parse import quote_plus
import json
import urllib.error
import base64
from func_timeout import func_timeout, FunctionTimedOut
from utils import *

BAIDU_API_KEY = "xxxxxx"
BAIDU_SECRET_KEY = "xxxxxx"
```

2、语音录制类（AudioRecorder）

定义该类的目的是实现录音功能，录制完成后将音频文件保存到本地，以供后续进行语音识别时使用。（1）开始录音使能：当按下空格键时，调用 **startRecord** 方法开始录音；（2）音频录制：通过回调函数 **AudioRecording**，将录制的音频帧添加到音频帧列表中；（3）停止录音并保存音频文件：当释放空格键时，调用 **stopRecord** 方法停止录音并将音频帧写入保存路径的文件。

该类的定义如下：

```
class AudioRecorder:
```

该类的各函数功能如下：

1) **__init__(self,savePath)**: 类的构造函数，初始化语音录制类的各个属性，并设置保存录音文件的路径。

```
def __init__(self, savePath):
    if savePath == None:
        self.savePath =
        "/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech
        /res/audio/record/temp.wav" # 录音文件存放地址
    else:
        self.savePath = savePath
        self.sampleRate = 16000 # 采样率,语音信号的频率范围通常更窄(低于
        8000Hz)
        self.channels = 1 # 音频声道数量
        self.width = 2 # 音频量化位数: 16 位
        self.driveIndex = 2 # 麦克驱动号
        self.chunk = 256 # 缓存中存放帧数
```

```

self.recording = False # 正在录音标志
self.frames = [] # 录音数据列表
self.finish = False # 录音结束标志
self.audio = pyaudio.PyAudio() # 创建音频类实例
self.stream = None # 创建音频流对象
keyboard.on_press_key(
    "space", lambda _: self.startRecord()) # 空格键按下开始录音
keyboard.on_release_key(
    "space", lambda _: self.stopRecord()) # 释放空格键停止录音
print("[Info] 按键录制音频程序启动!")

```

2) **AudioRecording(self, audio, count, time, status)**: 音频录制的回调函数，在录音过程中被调用用于将音频帧添加到列表中。

```

def AudioRecording(self, audio, count, time, status):
    """
    音频录制
    """
    if self.recording:
        self.frames.append(audio) # 将音频帧添加到列表中
    return (audio, pyaudio.paContinue)

```

3) **startRecord(self)**: 开始录音使能的方法，当按下空格键时调用。清空音频帧列表、设置录音完成标志为 **False**，并创建音频流对象进行录音。（全部代码请参考源文件）

```

def startRecord(self):
    """
    开始录音使能
    """

```

4) **stopRecord(self)**: 停止录音并保存音频文件的方法，当释放空格键时调用。停止音频流的录制、关闭音频流、将音频帧写入保存路径的文件，并将录音完成标志设置为 **True**。

```

def stopRecord(self):
    """
    停止录音并保存音频文件
    """

```

3、播放器类（AudioPlayer）

播放器为智能交互过程中的辅助工具，会播放本地的音频文件。在该实训对应的 `speech.py` 代码文件“**AudioPlayer**”类包含有完整的音频播放器的功能，音频播放器包含有如下一些基础功能：回车键播放音乐、暂停播放音乐、停止播放音乐、增加音量、减少音量、切换上一首、切换下一首、播放广播音频以及自动恢复音乐播放(广播完成后)等，相关代码较多，本部分只介绍类中以及函数的各部分功能，理解过程需要对照代码结合查看。在语音

交互过程中，会用到该类的部分函数功能。

第一步，音频播放器 `AudioPlayer` 类初始化，在类的初始化方法 `__init__` 中，会传入一个参数 `musicFolder`，用于指定音乐文件的存放目录。如果 `musicFolder` 为空，则默认设置为 `None`，否则就使用传入的值作为音乐文件存放目录。

类的属性包括：

`musicIndex`：音乐播放序号；

`playlist`：音乐播放列表，通过 `os.listdir` 函数获取音乐目录中的文件列表；

`musicing`：音乐播放状态，初始设置为 `False`；

`radioing`：当前播放内容，默认为广播；

`musicPos`：记录音乐播放进度，初始设置为 0；

`enable`：播放器使能，通过检查播放列表长度判断是否大于 0，初始设置为 `False`；

`volume`：音量，初始设置为 0.5；

`pause`：音频是否人为操作暂停，初始设置为 `False`。

在初始化方法中，还设置了键盘按键的回调函数，用于控制音乐播放。例如，按下右箭头键会触发 `nextMusic()`（下一首）方法，按下左箭头键会触发 `previousMusic()`（前一首）方法，以此类推。

初始化方法还包括系统播放器的初始化（使用了 `pygame.init()` 函数）和设置初始音量。

（全部代码请参考源文件）

```
class AudioPlayer:
    """
    音频播放器
    """

    def __init__(self, musicFolder):
        if musicFolder == None:
            self.musicFolder = None # 音乐文件存放目录
        else:
            self.musicFolder = musicFolder
        self.musicIndex = 0 # 音乐播放序号
        self.enable = False # 播放器使能：播放列表>0
        self.musicing = False # 音乐播放状态
        self.radioing = False # 当前播放内容：广播
        self.musicPos = 0 # 记录音乐播放进度
        .....
```

第二步，了解各部分函数功能。

① 函数 `setVolume` 是设置音量的方法，设置播放音量从 0.0~1.0。它接受一个参数 `volume`，用于设置音量值。

② 函数 `playMusic` 是播放音乐的方法，播放音乐的方式为随机播放。

③ 函数 `pauseMusic` 是暂停音乐的方法。

④ 函数 `resumeMusic` 是恢复播放音乐的方法。

⑤ 函数 `stopMusic` 是停止播放音乐的方法。

⑥ 函数 `nextMusic` 是切换到下一首音乐的方法，`previousMusic` 是切换到上一首音乐的方法。

⑦ `turnUpVolume` 函数为增加播放音量功能，`turnDownVolume` 函数为降低播放音量功能。

⑧ `playRadio` 播放广播音频函数，该功能的主要任务就是加载语音合成后的文件，用于播放音频所用。

⑨ `autoResumeMusic` 自动恢复音乐播放函数。

4、语音识别类（AsrOnline）

智能机器人需要联网调用云端的语音识别 API 实现语音识别功能。该类的作用是对上传的音频文件进行语音识别转化为文本，定义如下：

```
class AsrOnline:
```

下面是该类中三个函数的作用：

1) 初始化函数：该函数定义了识别语音时需要用到的基本信息，包括 API 密钥、文件位置、功能类型等。

```
def __init__(self, audioPath, audioPlayer):
    self.apiKey = BAIDU_API_KEY # 百度语音 API 密钥
    self.secretKey = BAIDU_SECRET_KEY # 百度语音安全密钥
    if audioPath == None:
        self.audioPath =
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_spe
ech/res/audio/record/temp.wav" # 录音文件存放地址
    else:
        self.audioPath = audioPath
    self.cuid = '123456PYTHON'
    self.scope = 'audio_voice_assistant_get' # 云端 API 语音功能类型
    self.audioPlayer = audioPlayer
    print("[Info] 在线语音识别功能已启动!")
```

2) 语音识别函数（`speechRecognition`）：该函数主要用于进行语音识别功能，识别前

需要判断是否和云端建立连接；连接建立成功后，将本地录制好的音频文件上传，上传前需要注意判断文件内容是否为空以及音频文件的具体格式；确认无误后发送请求，将音频文件上传到云端进行识别，识别后云端会返回数据包；本地接收到数据包后，进行解析，解析后即可输出识别的内容，如果获取数据失败，则会报错。

```
def speechRecognition(self):
    """
    语音识别(调用本地音频录制文件)
    return: str: 识别结果
    """
```

3) 云端验证信息

在语音识别前，我们需要先确保和云端的连接是正常的，这就需要设计验证功能，我们将 API 信息打包处理后即可发送至云端，然后接收云端返回的数据，如果连接失败则出现错误提示，连接成功则检查用户信息，确认无误则返回访问令牌，否则会报错。

```
def fetchToken(self):
    """
    获取云端 token 信息
    return: False: 账户检验失败, str: token 信息
    """
```

5、关键词匹配类 (WordsMatching)

该类主要功能是用于实现字符串检索功能，基于 interaction.json 文件进行匹配。定义如下：

```
class WordsMatching:
```

该类的主要功能和各函数的作用如下：

1) `__init__(self, jsonPath)`: 类的构造函数，初始化字符串检索类的各个属性。根据传入的 JSON 文件路径初始化 `jsonPath` 属性，并读取该 JSON 文件，将其内容加载到 `jsonStream` 属性中。如果指定的 JSON 文件路径错误，打印错误信息。

```
def __init__(self, jsonPath):
    if jsonPath == None:
        self.jsonPath =
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech
/res/config/interaction.json" # Json 文件路径
    else:
        self.jsonPath = jsonPath # Json 文件路径
        self.jsonStream = None # Json 数据流
        if os.path.exists(self.jsonPath):
            with open(self.jsonPath, 'r') as file: # 打开 Json 文件
```

```

        str = file.read()
        self.jsonStream = json.loads(str)
    else:
        print("[Error] 字符匹配 Json 文件路径错误!")
        return

```

2) **wordsMatching(self, str)**: 字符串检索方法, 用于根据关键字匹配对应的内容。首先判断是否成功加载了 JSON 数据, 如果未加载成功则直接返回。然后遍历 JSON 数据中的 chat 对话类型, 对每个关键字进行检查, 判断传入的字符串是否包含关键字。如果包含关键字, 则随机选择一个对应的回复内容, 返回类型为"chat"和对应的回复内容。

```

def wordsMatching(self, str):
    """
    字符检索(关键字)
    return: json[type][item]
            1) chat,response
            2)      ,str
    """

```

最后, 如果没有匹配到任何内容, 则返回空字符串。

3) 语音配置文件 “interaction.json”

我们可以根据本地的 “interaction.json” 去自行配置识别的关键词与对应回复的内容。在根目录文件夹下依据路径 “../res/config/interaction.json” 找到该配置文件, 格式如下图所示。

```

{,
  "/*-----[自我介绍]-----*/": "",
  "hello": {
    "keyword": [
      "你好",
      "嗨",
      "Hello",
      "您好"
    ],
    "response": [
      "你好, 欢迎使用 商服机器人!",
      "你好, 很高兴为您服务!"
    ]
  },
}

```

图 6-13 语音配置文件 “interaction.json”

“问候和自我介绍”是一个对话类型, 用于表示问题和回复的主题。①“keyword”是关键字列表, 包含了可以触发该对话类型的关键词。在这个例子中, 关键词是“Hello”、“你好”和“您好”。②“response”是回复列表, 包含了与上述关键词匹配时可以随机选择的回复内容。在这个例子中, 有两个回复。

当用户输入的字符串包含其中一个关键词时，程序会返回该对话类型为"chat"，并从回复列表中随机选择一个回复返回给用户。例如，如果用户输入"Hello"，程序可能会返回"你好，很高兴为您服务！"作为回复。

对照上述代码数据的格式，我们可以自定义关键词与回复内容，例如我们希望增加对于“张三”的关键词匹配与回复内容。代码格式就可以设定为如下所示，将代码插入到“逗号”之后，按照上述格式进行内容生成。

```
"张三介绍": {
    "keyword": [
        "张三",
        "张三是谁",
        "谁是张三",
        "张三是哪位"
    ],
    "response": [
        "张三是罗翔口中经常举例的那个人，他是作为事件主人公的代指人物名称"
    ]
}
```

注意：本实训只演示自定义方法，所提供代码中并不包含“张三”这一内容的关键词匹配与内容回复，同学们可以自行添加关键词匹配内容与语音回复内容进行验证。

6、语音合成类（TtsOnline）

该类主要功能是实现文字合成语音的功能，使用的是百度在线语音库。定义如下：

```
class TtsOnline:
```

该类的主要功能和各函数的作用如下：

1) `__init__(self, audioPath)`: 类的构造函数，初始化文字合成语音类的各个属性。根据传入的语音合成文件存放地址初始化`audioPath`属性，并设置百度语音 API 密钥和安全密钥等参数。

```
"""
文字合成语音(百度在线语音库)
"""

def __init__(self, audioPath):
    if audioPath == None:
        self.audioPath =
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech
/res/audio/tts/temp.mp3" # 语音合成文件存放地址
    else:
        self.audioPath = audioPath # 语音合成文件存放地址
```

```

self.apiKey = BAIDU_API_KEY # 百度语音 API 密钥
self.secretKey = BAIDU_SECRET_KEY # 百度语音安全密钥
self.cuid = '123456PYTHON'

self.scope = 'audio_voice_assistant_get' # 云端 API 语音功能类型
self.person = 0 # 播音员: 1) 基础音库: 0: 度小美, 1: 度小宇, 3 为度逍遥,
4 为: 度丫丫;
# 2) 精品音库: 5: 度小娇, 103: 度米朵, 106: 度博文, 110: 度小童, 111: 度小萌,
默认: 度小美
self.speed = 6 # 语速, 取值 0-15, 默认为 5 中语速
self.pitch = 5 # 音调, 取值 0-15, 默认为 5 中语调
self.volume = 5 # 音量, 取值 0-9, 默认为 5 中音量
self.format = 3 # 语音文件格式 (3: mp3 (default) 4: pcm-16k 5: pcm-8k
6. wav)

print("[Info] 在线语音合成功能已启动!")

```

2) **fetchToken(self)**: 获取云端 token 信息的方法, 用于向百度云发送请求获取访问令牌 token。根据设置的 API 密钥和安全密钥, 向百度云发送请求获取 token 信息。

```

def fetchToken(self):
    """
    获取云端 token 信息
    return: False: 账户检验失败, str: token 信息
    """
    params = {'grant_type': 'client_credentials',
              'client_id': self.apiKey,
              'client_secret': self.secretKey}
    request = urlencode(params).encode('utf-8')
    response = Request('http://aip.baidubce.com/oauth/2.0/token',
                       request) # 获取云端 API 信息

```

3) **speechTransfer(self, text)**: 在线语音合成的方法, 用于将文本转换为语音。首先调用`fetchToken`方法获取访问令牌 token, 如果获取失败则返回 False。然后对文本进行编码并根据设置的参数构建请求参数。将请求发送到百度云, 如果请求成功且返回的内容为音频文件, 则将其保存到指定的语音合成文件存放地址, 返回文件路径。如果合成失败, 则返回 False。

```

def speechTransfer(self, text):
    """
    在线语音合成
    """

```

该类提供了使用百度在线语音库进行文字合成语音的功能。用户可以通过调用 **speechTransfer** 方法, 传入需要合成的文本, 获取合成后的语音文件路径。

除此之外, 该部分代码还有可以结合上述所有类完成语音交互的主函数程序, 此处代码

逻辑不过多进行赘述。可以在外部打开终端界面，输入指令“`python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/scripts/speech.py`”启动语音识别、语音合成相关功能。

步骤 4：智能交互代码逻辑（aiInteraction.py）

此部分将步骤 3 中涉及的语音技术与智能机器人本身进行了交互控制，并且生成了相应的 UI 界面，语音控制机器人进行软件功能启动/关闭或其它功能。该代码逻辑中包含有一个智能交互主控类与一个主程序，主控类中包含有初始化功能函数、判断交互类型功能函数、执行交互功能函数、键盘控制功能启动功能函数、控制语音输出功能函数、主题 UI 显示语音对话内容功能函数、停止语音交互进程功能函数；以及机器人控制类与功能启动类两个子类。

1、导入函数库

- （1）引入 `time` 库：用于处理时间相关的操作，例如获取当前时间、延时等。
- （2）引入 `speech` 库：自建函数功能包，用于语音识别、语音合成、关键词匹配、音频播放器等功能。
- （3）引入 `uart` 库：自建函数功能包，这个库用于与硬件设备进行串口通信，通过串口与硬件设备进行数据交互。
- （4）引入 `PIL` 库：强大的图像处理库，提供了读取、绘制、处理图像等功能。
- （5）引入 `numpy` 库：用于科学计算和数值运算的库，经常与图像处理和矩阵运算相关的操作一起使用。
- （6）引入 `datetime` 库：用于处理日期和时间相关的操作，例如获取当前日期和时间、时间格式化等。
- （7）引入 `os` 库：提供了操作系统相关的功能。
- （8）引入从 `cv2` 库：计算机视觉库，可以进行图像处理、特征提取、识别等相关操作。
- （9）引入 `signal` 库：用于处理系统信号，可能用于捕获程序中断信号或其他与系统相关的信号的处理。

```
import time
from speech import *
from uart import *
from PIL import Image, ImageDraw, ImageFont
import numpy as np
```

```
from datetime import datetime
import os
import cv2
import signal
```

2、智能交互主控类

该类用于集成智能交互过程中所需要的各个函数功能，接下来对该类的代码功能进行概要介绍。

```
class AiInteraction:
    """
    智能交互主控类
    """
```

(1) 初始化函数（__init__(self)）

以下是对该类中每个方法和属性的解释，此部分为初始化部分，代码过长，代码过程详看源代码。

`__init__()`方法：类的初始化方法，初始化了许多对象和变量。具体包括：

- 1) `audioRecorder`：音频录制对象，用于录制用户的语音输入。
- 2) `audioPlayer`：音频播放对象，用于播放音乐。
- 3) `asrOnline`：在线语音识别对象，用于将语音转换为文字。
- 4) `ttsOnline`：文字转语音对象，用于将文字转换为语音。
- 5) `wordsMatching`：字符串匹配对象，用于检索特定字符串。
- 6) `soundFolder`：广播提示音文件夹的路径。
- 7) `fontBland`：对话内容显示文字的字号。
- 8) `fontCH`：中文字体对象。
- 9) `uart`：串口对象，用于与设备进行通信。
- 10) `control`：机器人控制类的实例。
- 11) `launch`：功能启动类的实例。
- 12) `running`：程序运行标志，表示程序是否在运行中。
- 13) `keyboardEnable`：键盘开启功能标志，表示是否开启了键盘控制功能。
- 14) `lenChatRow`：UI 界面中每行显示的字数。
- 15) `chat`：模拟语音输入，存储用户的语音输入内容。
- 16) `word`：显示对话内容。

最后，完成上述所有初始化操作以后，清除缓存和关闭 ROS 进程的操作。

(2) 判断交互类型函数 (decide(self))

这段代码定义了一个名为 `decide` 的方法，用于判断交互类型。它根据录音是否完成或是否开启键盘控制来决定采取何种交互方式。具体实现如下：

1) 如果录音已完成或已开启键盘控制：清空录音完成标志。

2) 如果开启了键盘控制，则将键盘控制输入的参数作为语音识别的结果，同时清空输入内容。否则，启动在线语音识别功能获取语音识别结果。

3) 如果语音识别结果长度大于 0：

如果结果中包含"结束"，则停止智能交互进程并返回空结果。

如果结果中包含"停止"，则模拟键盘输入字母"c"关闭 Python 程序，停止播放音乐，并返回命令类型和相应的指令结果。

如果结果中包含"向"、"自检程序"或"氛围灯"，则返回控制类型和结果。

如果结果中包含"保存"或"地图"，则返回命令类型和相应的指令结果。

如果结果中包含"电量"，则获取电压和电量信息，并返回对话类型和当前电池电量和电压信息。

如果结果中包含"几点"/"时间"或"几号"/"日期"，则返回对话类型和当前时间或日期信息。

如果结果中包含"音乐"，则开始播放音乐（随机开始），并返回空结果。

如果结果中包含"上一首"/"下一首"，则播放上一首/下一首音乐，并返回空结果。

否则，通过字符串检索功能检索特定关键字，并返回对话类型和响应结果。

4) 如果未开启键盘控制且语音识别结果无效，则返回类型为"0" (unknown) 和响应为"unknown.mp3"的结果。

最后，如果录音未完成且未开启键盘控制，则返回空结果。

该方法的作用是根据用户的语音输入或键盘输入，判断用户的意图和需求，并返回相应的交互类型和响应结果。(完整代码请参考源文件)

```
def decide(self):
    """
    判断交互类型
    """
    if self.audioRecorder.finish or self.keyboardEnable: # 录音完成
        或者已开启键盘控制
        self.audioRecorder.finish = False # 清空录音完成标志
        if self.keyboardEnable:
            # 将键盘控制传入的参数作为语音识别的结果 (会关闭语音识别)
            res = self.chat
```



```

        self.chat = "" # 清空输入内容
    else:
        res = self.asrOnline.speechRecognition() # 启动在线语音识别
    if len(res) > 0: # 语音识别结果有效
        if res.__contains__("结束"):
            self.stop(0, 0) # 结束智能交互进程
            return "", ""
        elif res.__contains__("停止"):
            .....

```

(3) 执行交互函数 (execute(self, type, response))

execute 函数用于执行交互操作。它根据交互类型和响应结果进行不同的操作，并返回需要合成的音频内容或音频文件名。根据 cmd、ctrl、chat、0 等几种自定义类型去执行不同的交互内容。

```

def execute(self, type, response):
    """
    执行交互
    """
    if type != "":
        if type == "cmd":
            self.word = response[0] # 显示文字
            res = self.launch.launch(response) # 启动功能
            response = res # 统一输出
        elif type == "ctrl":
            self.word = response # 显示文字
            response = self.control.motionControl(response) # 控制机器人
        elif type == "chat":
            self.word = response
        elif type == "0":
            self.word = "抱歉我没有听清，请您再讲一次"
    print(self.word) # 显示语音交互内容
    # 返回需要合成音频的文字或音频文件名称(.mp3)
    return response

```

(4) 键盘控制功能函数 (keyboardControl(self, value))

keyboardControl 的方法，用于处理键盘控制功能。它根据接收到的键盘输入值，对不同按键进行不同的操作。具体实现如下：

第一步，键盘功能开启与关闭切换。

如果输入包含 "tab down": 表示按下了按键 1: 清空语音输入内容，切换键盘开启功能标志，打印 "键盘控制已开启"，播放 "turnOn.mp3" 语音提示。否则，打印 "键盘控制已关闭"，并播放 "turnOff.mp3" 语音提示。

第二步，根据键盘按下的按键进行功能交互。

如果输入包含 "(1 down": 表示按下了按键 1, 将语音输入内容设置为 "启动建图功能"。

如果输入包含 "(2 down": 表示按下了按键 2, 将语音输入内容设置为 "启动导航功能"。

如果输入包含 "(3 down": 表示按下了按键 3, 将语音输入内容设置为 "启动仿真建图功能"。

如果输入包含 "(4 down": 表示按下了按键 4, 将语音输入内容设置为 "启动仿真导航功能"。

如果输入包含 "(5 down": 表示按下了按键 5, 将语音输入内容设置为 "启动智能安控功能"。

如果输入包含 "(6 down": 表示按下了按键 6, 将语音输入内容设置为 "启动垃圾巡检功能"。

如果输入包含 "(7 down": 表示按下了按键 7, 将语音输入内容设置为 "启动姿态估计功能"。

如果输入包含 "(8 down": 表示按下了按键 8, 将语音输入内容设置为 "启动智能跟随功能"。

如果输入包含 "(0 down": 表示按下了按键 0, 将启动自检程序。

如果输入包含 "s down" 或 "S down": 表示按下了按键 s, 将语音输入内容设置为 "保存地图"。

如果输入包含 "k down" 或 "K down": 表示按下了按键 k, 将语音输入内容设置为 "停止 ROS 功能"。

如果输入包括"a down"、"d down"、"w down"、"x down": 表示控制智能机器人前后左右四个方向移动。

如果输入包含 "esc down": 表示按下了按键 esc, 将语音输入内容设置为 "结束智能交互"。

该方法用于响应键盘输入, 并根据按下的按键赋值不同的语音输入内容, 以实现对机器人的不同功能控制。(完整代码请参考源文件)

```
def keyboardControl(self, value):  
    """  
    键盘控制功能启动  
    """  
    value = str(value) # 转化为字符串型
```

```

if value.__contains__("tab down"): # 如果按键1被按下
    self.chat = "" # 清空按键输入
    self.keyboardEnable = not self.keyboardEnable # 键盘开启功能标志
    if self.keyboardEnable:
        print("键盘控制已开启")
        aiInteraction.speechControl("turnOn.mp3") # 播放语音
    else:
        print("键盘控制已关闭")
        aiInteraction.speechControl("turnOff.mp3") # 播放语音

if value.__contains__("1 down"): # 如果按键1被按下
    self.chat = "启动建图功能"
elif value.__contains__("2 down"): # 如果按键2被按下
    self.chat = "启动导航功能"
elif value.__contains__("3 down"): # 如果按键3被按下
    self.chat = "启动仿真建图功能"
elif value.__contains__("4 down"): # 如果按键4被按下
    self.chat = "启动仿真导航功能"
elif value.__contains__("5 down"): # 如果按键5被按下
    self.chat = "启动智能安控功能"
elif value.__contains__("6 down"): # 如果按键6被按下
    self.chat = "启动垃圾巡检功能"
elif value.__contains__("7 down"): # 如果按键7被按下
    self.chat = "启动姿态估计功能"
elif value.__contains__("8 down"): # 如果按键8被按下
    self.chat = "启动智能跟随功能"
elif value.__contains__("0 down"): # 如果按键0被按下
    self.chat = "启动自检程序"
elif value.__contains__("s down") or value.__contains__("S down"): # 如果按键s被按下
    self.chat = "保存地图"
elif value.__contains__("k down") or value.__contains__("K down"): # 如果按键k被按下
    self.chat = "停止所有程序"
elif value.__contains__("a down") or value.__contains__("A down"): # 如果按键A被按下
    self.chat = "向左转"
elif value.__contains__("d down") or value.__contains__("D down"): # 如果按键D被按下
    self.chat = "向右转"
elif value.__contains__("w down") or value.__contains__("W down"): # 如果按键W被按下
    self.chat = "向前移动"

```

```

elif value.__contains__("x down") or value.__contains__("X
down"): # 如果按键 x 被按下
    self.chat = "向后移动"
elif value.__contains__("esc down"): # 如果按键 esc 被按下
    self.chat = "结束智能交互"

```

(5) 控制语音输出功能函数 (speechControl(self, response))

该方法用于控制机器人的语音输出功能。它可以根据响应结果的类型播放相应的音频文件或进行在线语音合成，并立即播放生成的语音。此外，它还可以自动切换音频播放状态，保证语音输出正常进行。

```

def speechControl(self, response):
    """
    控制语音输出
    param
    """
    if response != "" and response is not None:
        if response.__contains__(".mp3"):
            response = self.soundFolder + response # 获取音频所在路径
            self.audioPlayer.playRadio(response) # 立即播放语音
        else:
            audioPath = self.ttsOnline.speechTransfer(
                response) # 在线语音合成 mp3
            if audioPath != False: # 语音合成成功
                self.audioPlayer.playRadio(audioPath) # 立即播放语音
                self.audioPlayer.autoResumeMusic() # 自动切换音频播放状态

```

(6) UI 显示函数 (showChat(self, img))

这个方法主要用于显示语音对话内容，包括以下几个部分：

1) 在图像上显示多行语音对话内容，根据内容长度进行限制，超过 35 个字符长度的部分用省略号表示。

2) 显示对话内容的位置是在 UI 图像底部的中间位置，使用绿色字体进行显示。

该方法目的是将语音对话内容以美观的方式展示在 UI 图像上，以便用户可以清晰地看到对话的内容。(代码请参考源文件)

(7) 停止语音交互进程的函数 (stop(self, signum, frame))

该方法的作用是当需要停止语音交互进程时，执行相应的操作并进行必要的清理工作，最后以适当的延时使程序暂停。具体的停止操作包括关闭 ROS 进程、停止串口通信和播放结束语音提示。

```

def stop(self, signum, frame):
    """

```

停止语音交互进程

```
"""
if self.running:
    os.system("echo 1 > /proc/sys/vm/drop_caches") # 清空缓存
    # 关闭所有 ROS 进程
    os.system("gnome-terminal -- killall -r ros rviz master")
    self.uart.stop(0, 0) # 关闭串口
    self.speechControl("finish.mp3") # 播放结束语
    self.running = False # 关闭语音交互进程
    time.sleep(4) # 延时四秒关闭程序
```

(8) 机器人控制子类 (class Control)

Control 类提供了以下控制功能：

- 1) 控制机器人的运动：向左转、向右转、向前移动、向后移动。
- 2) 启动机器人的自检程序。
- 3) 控制机器人 RGB 氛围灯的开启和关闭。

通过调用 Control 类的方法，可以实现对机器人的运动和功能进行控制。这些功能可以根据输入的响应结果来执行相应的操作，并返回相应的音频文件名，用于语音提示。

```
class Control:
    """
    机器人控制类
    """

    def __init__(self, uart):
        self.uart = uart # 串口通信

    def motionControl(self, response):
        """
        运动控制
        """
```

(9) 功能启动子类 (class Launch)

根据传入的响应结果来判断要启动还是停止 ROS 功能。

第一步，如果响应结果中包含“启动”的指令，并且已经开启了应用（即 ROS 功能已经启动），则返回“当前已开启应用，请您关掉当前应用后重试”。

第二步，如果响应结果中包含“启动”的指令，并且应用未开启，则关闭 Python 串口，启动 ROS 功能，并更新应用状态。

第三步，如果响应结果不包含“启动”的指令，则如果串口未运行，重新开启 Python 串口，关闭当前功能，并更新应用状态。

第四步，最后返回音频文件路径。

通过调用 Launch 类的方法，可以根据用户的指令来启动或关闭指定的 ROS 功能，并返回相应的音频文件路径，用于语音提示。

```
= class Launch:
    """
    功能启动类
    """
    def __init__(self, control):
        self.running = False # 是否启动 ROS 功能标志
        self.ctrl = control # 获取对机器人的控制
    def launch(self, response):
        """
        启动 ROS 功能
        """
```

3、主程序（缺失代码需填空）

此部分为语音交互的主程序，会结合前文所讲述的各个类功能进行综合运用，完成语音合成的功能任务。主程序根据代码功能逻辑主要分为两部分，第一部分是各个类功能的初始化，第二部分是程序运行起来完成语音交互的功能。

（1）初始化部分代码讲解（代码填空）

这段代码的功能是启动智能交互程序。它实例化智能交互控制类，并通过读取本地视频、播放启动语音来显示智能交互的 UI 界面，并监听键盘输入来进行相应的功能控制。当切换到键盘模式时，会从语音交互 UI 界面切换到键盘 UI 交互界面。

```
if __name__ == "__main__":
    """
    主程序
    """
    aiInteraction = AiInteraction() # 实例化智能交互主控类
    signal.signal(signal.SIGINT, aiInteraction.stop) # 定义软件退出信号量

    # 显示智能交互 UI
    # ##### 代码填空开始处 ##### #
    """
    代码填空提示：读取本地视频，开启 UI 界面
    1. 通过 cv2 调用 VideoCapture 函数功能；
    2. 根据路径
    /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/
    res/image/aiInteraction.mp4 加载视频
    3. 需要使用变量 capture 去承接调用结果。
    """
```

```

# ##### 代码填空结束处 ##### #
keyboardImg =
cv2.imread("/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/res/image/keyboard.png") #加载操作指示图
aiInteraction.speechControl("aiInteraction.mp3") # 播放语音
capture.set(cv2.CAP_PROP_FPS, 60) # 设置视频的读取速率
capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
cv2.namedWindow("Video", cv2.WINDOW_NORMAL) # 创建 UI 窗口
cv2.setWindowProperty('Video', cv2.WND_PROP_FULLSCREEN,
cv2.WINDOW_FULLSCREEN) # 将窗口铺满屏幕
keyboard.on_press(
lambda key: aiInteraction.keyboardControl(key)) # 创建一个线程监听键盘

```

(2) 语音交互逻辑（代码填空）

该部分片段描述了程序的主要运行循环。在循环中，检测是否有功能开启并相应地进行处理，包括循环读取本地视频用于显示语音交互的 UI 界面，处理交互类型并执行相应操作，播放语音回复，并通过按下 ESC 键来停止智能交互。最后，释放资源并结束程序的运行。

```

while aiInteraction.running:
    if not aiInteraction.launch.running: # 如果未开启功能（开启功能时自动屏蔽智能交互界面）
        if not aiInteraction.keyboardEnable: # 如果开启了键盘控制使能则打开操作指示图
            # 读取视频数据
            # ##### 代码填空开始处 ##### #
            """
            代码填空提示：代码只有一行，读取视频数据
            1. 读取的数据一共有两个，分别用变量 ret 与 img 去获得。
            2. 通过 capture 调用 read 函数去读取视频数据；
            3. read 函数参数为空。

            """
            # ##### 代码填空结束处 ##### #
            if ret:
                if len(aiInteraction.word) > 0:
                    img = aiInteraction.showChat(img) # 显示语音回答内容
                    img = aiInteraction.showErrorCode(img) # 显示错误代码
                    cv2.imshow("Video", img) # 显示语音交互 UI 界面
                else:
                    capture =
cv2.VideoCapture("/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/res/image/aiInteraction.mp4") # 读取视频
cv2.namedWindow("Video", cv2.WINDOW_NORMAL) # 创建 UI 窗口

```



```

        cv2.setWindowProperty('Video',
cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN) # 将窗口铺满屏幕
    else:
        img = keyboardImg.copy() # 复制操作指示图
        cv2.imshow("Video", img) # 显示语音交互 UI 界面
        cv2.namedWindow("Video", cv2.WINDOW_NORMAL) # 创建 UI 窗口
        cv2.setWindowProperty('Video', cv2.WND_PROP_FULLSCREEN,
cv2.WINDOW_FULLSCREEN) # 将窗口铺满屏幕

    else:

        capture.release() # 释放摄像头
        cv2.destroyAllWindows() # 释放窗口

    aiInteraction.control.uart.transmitSysHeart() # 发送心跳信号
    type, response = aiInteraction.decide() # 判断交互类型
    response = aiInteraction.execute(type, response) # 执行交互
    aiInteraction.speechControl(response) # 播放语音

    if cv2.waitKey(1) == 27: # 按ESC退出程序
        aiInteraction.stop(0, 0)

aiInteraction.uart.stop(0, 0) # 关闭串口
capture.release() # 释放摄像头
cv2.destroyAllWindows() # 释放窗口

```

步骤 5：智能机器人语音交互实操效果运行

第一步，将机器人开机启动，将 PC 与机器人通过网线和以太网设置等步骤建立远程连接，基于 PC 上的 VSCode 打开机器人工程文件，根据路径“sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/scripts/”找到语音技术的源码文件 speech.py，在源代码中填入我们在云端申请的密钥。注意此处修改的是 py 文件，无需重新编译。

```

27 # 百度语音API密钥
28 BAIDU_API_KEY = " "
29 # 百度语音安全密钥
30 BAIDU_SECRET_KEY = " "

```

图 6-14 填写 API

第二步，在机器人主界面右上角连接本地 WIFI 实现机器人联网。

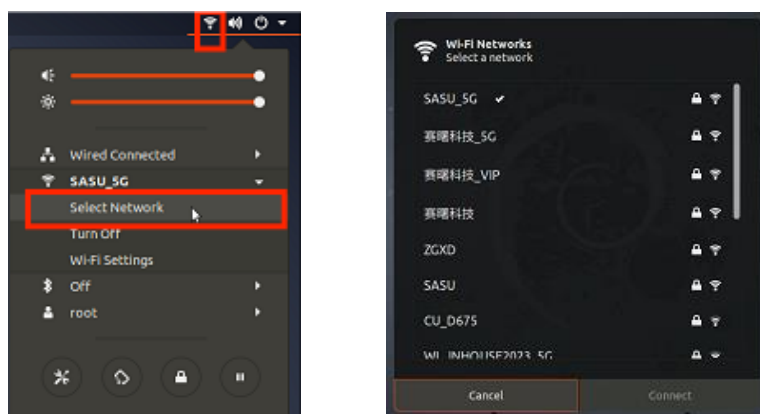


图 6-15 连接 WIFI

第三步，启动语音交互程序。

在智能机器人（不是个人 PC）的操作系统内打开终端，输入以下指令，用于执行对应的语音交互程序。

```
python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot  
_speech/scripts/aiInteraction.py
```

```
root@paddlepi:~# python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src  
/sebot_speech/scripts/aiInteraction.py
```

图 6-16 指令的执行

```
pygame 2.5.2 (SDL 2.28.3, Python 3.6.9)  
Hello from the pygame community. https://www.pygame.org/contribute.html  
[Info] 按键录制音频程序启动!  
[Info] 音乐播放程序启动!  
[Info] 在线语音识别功能已启动!  
[Info] 在线语音合成功能已启动!  
[Info] uart receive thread start!
```

图 6-17 语音交互启动

语音交互程序启动，系统自动出现下图语音交互 UI 界面，可查看语音交互的功能分类。

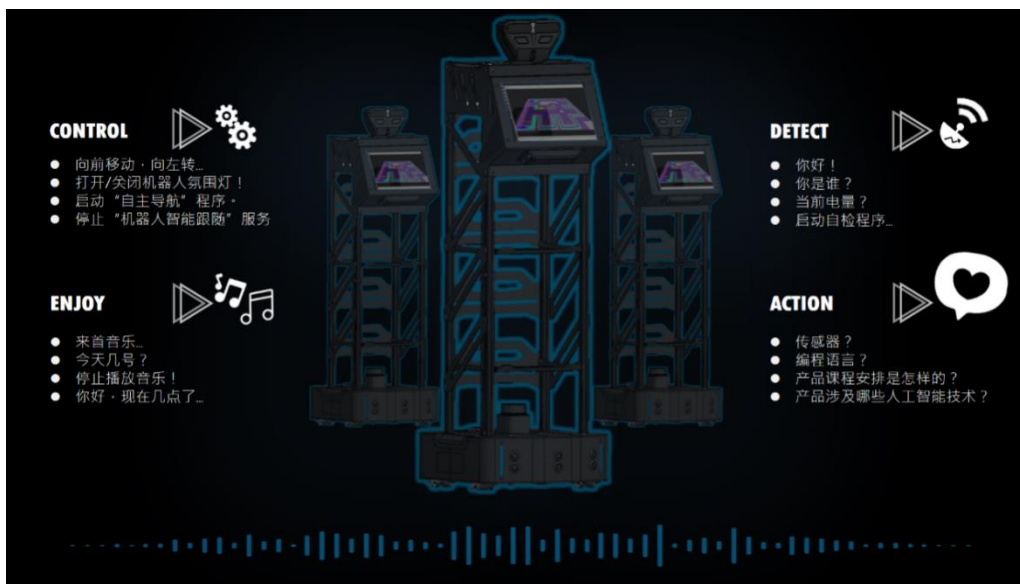


图 6-18 进入语音交互的 UI 界面

第四步，语音对话与语音控制机器人。

程序运行后，按住无线键盘的“空格”按键，即可开启录音功能。输入语音后，松开“空格”按键则表明录音完成。录音完成后等待智能机器人反馈回来的声音效果，同时在 UI 界面上会显示语音合成的文本，即机器人语音回答的文字内容，并且如果匹配到相应的机器人操作，比如“开启语音交互”，机器人则会按照语音输入的指令启动或停止某项软件功能，完成任务。

可以参考前面交互控制代码逻辑内容，通过语音控制机器人启动或停止某项功能，比如说出“启动建图功能”，则能看到机器人建图功能启动，Rviz 出现，接下来就可以用键盘或手柄控制机器人沿着地图走动开始建图了；或者说出“启动垃圾巡检功能”、“启动仿真导航功能”，都能启动相关程序，类似我们通过电脑键盘输入某个指令开启或停止相关功能。还可以参考下面 UI 界面，语音控制机器人其他运动、播放音乐、询问时间、咨询产品或课程安排或开启自检等等。

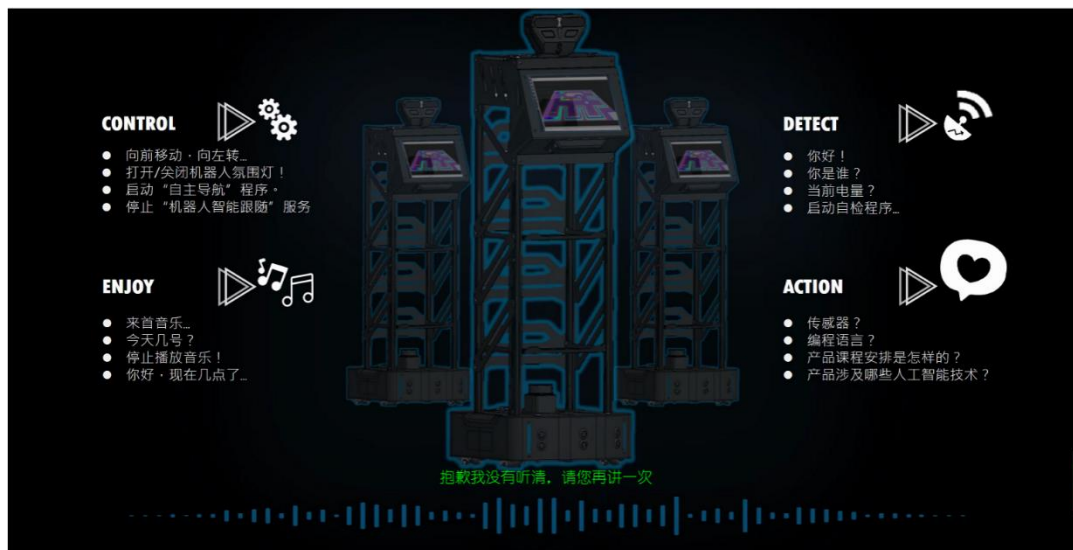


图 6-19 “语音交互” 进行中

第五步, 切换键盘控制页面。语音交互是人与智能机器人进行的一种交互方式, 使用者还可以通过键盘进行机器人控制。按下 TAB 按键, 切换到键盘控制页面, 查看一些键盘中的按键操作。如下图的键盘操作示意图我们在代码环节中也有讲解, 所以按照该页面中的提示进行操作控制即可。

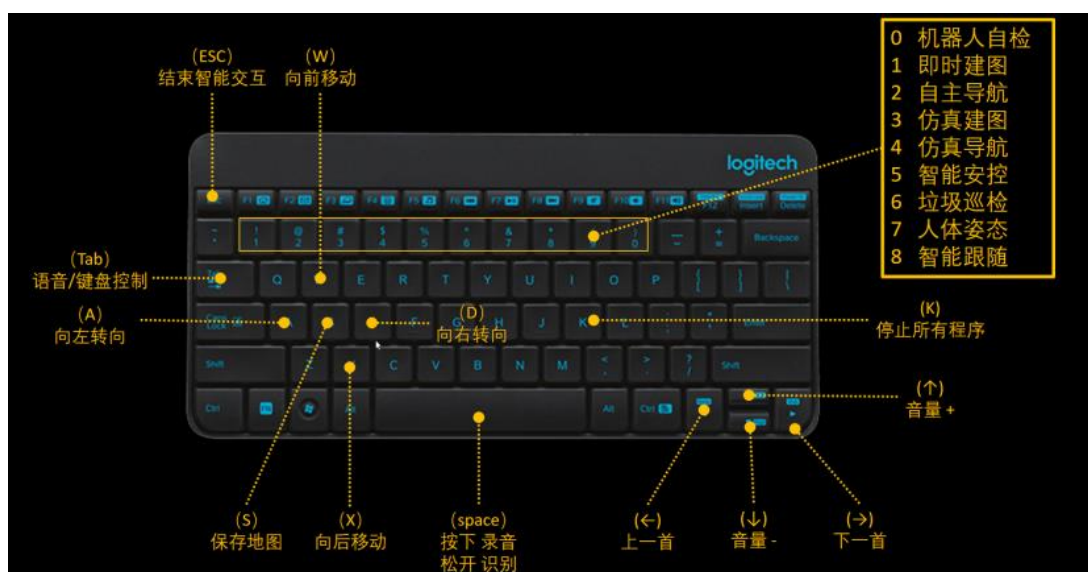


图 6-20 “键盘控制” 页面

第六步, 结合语音控制与键盘控制这两种控制方式可以完成对智能机器人的控制任务。例如语音输入“启动遥控建图程序”, 键盘控制按下按钮“1”启动即时建图程序。

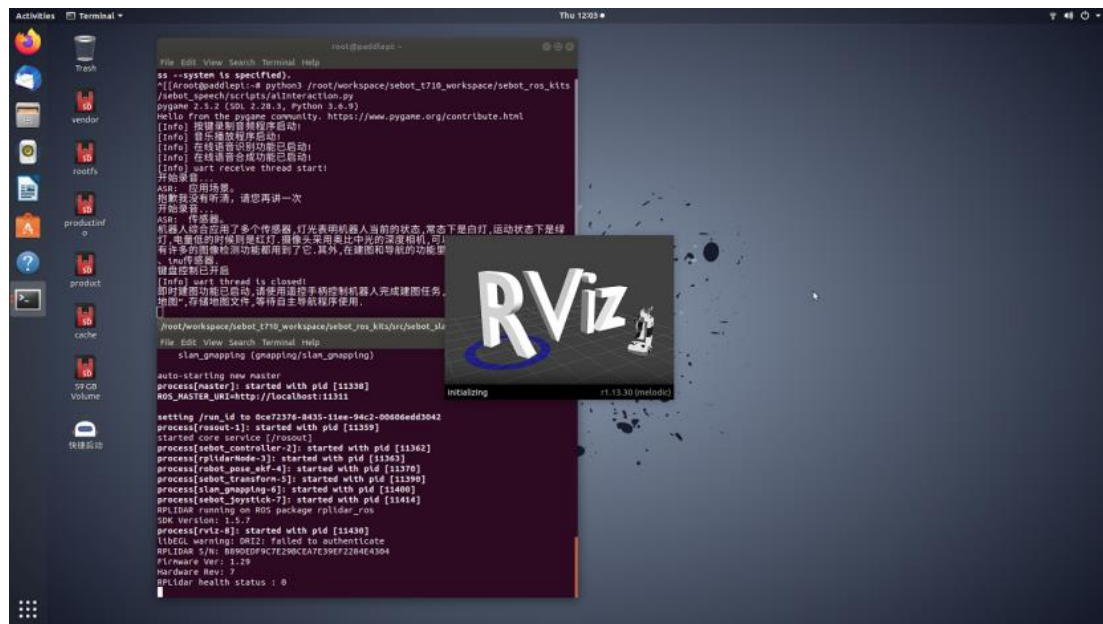


图 6-21 通过语音/按键启动建图功能

建图功能页面如下图所示。

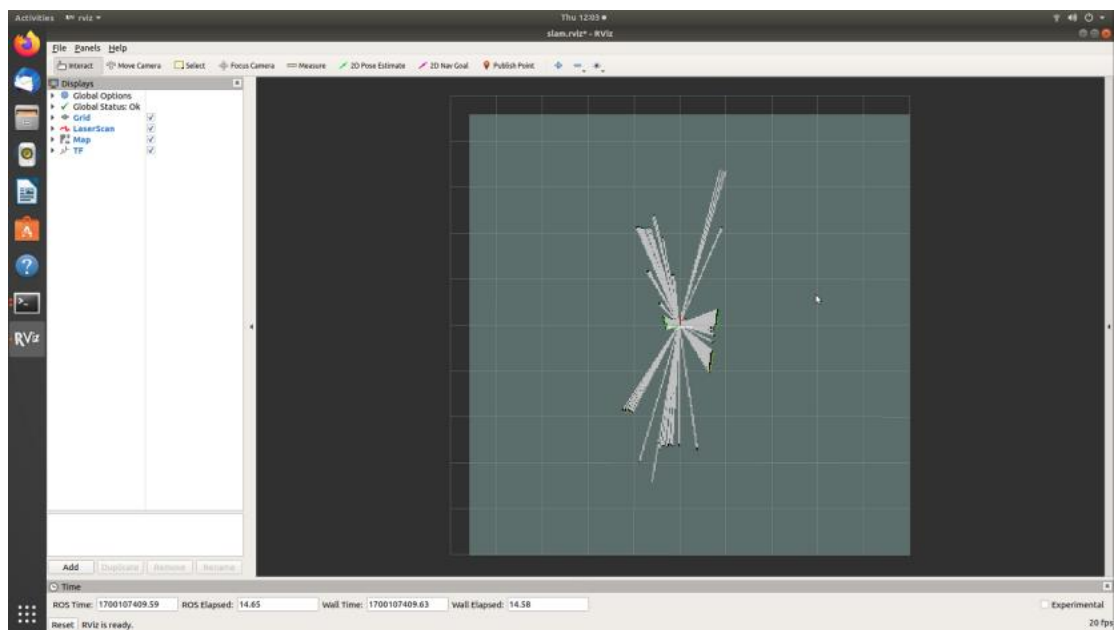


图 6-22 建图功能界面

在开启的对应功能界面按 K（按键模式）或者说“停止所有程序”（语音模式）即可停止所有功能，回到语音交互界面。最后，按 Esc（按键模式）或说“结束智能交互”（语音模式）即可退出智能交互功能。

注意事项：智能机器人有自检功能，如果要运行这个功能得先把智能机器人悬空，否则智能机器人会移动。